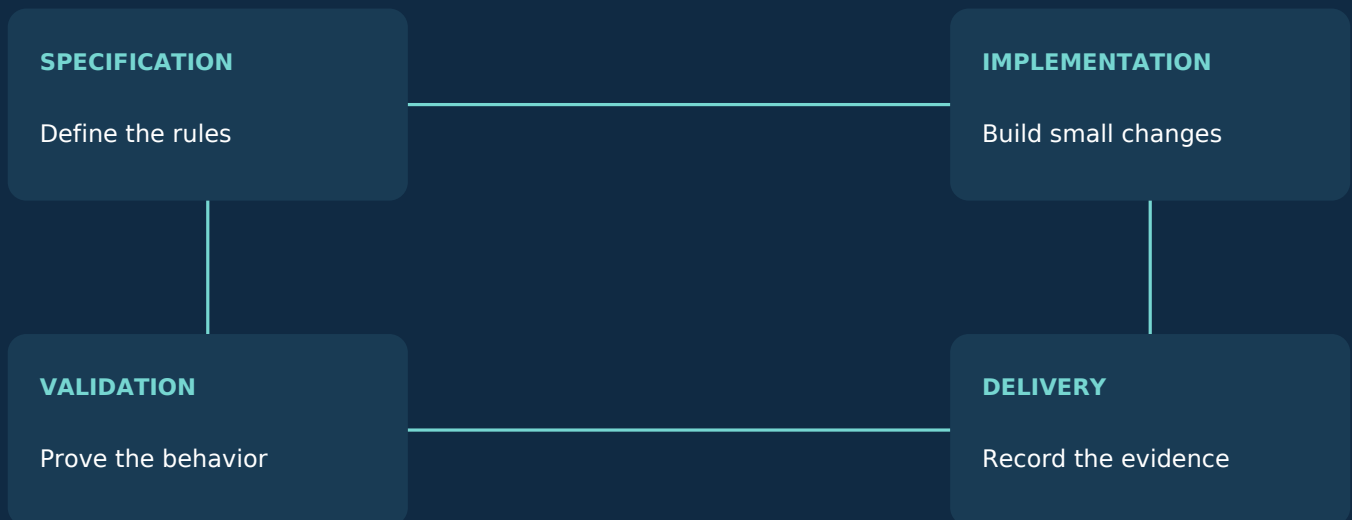


Coding with AI

From a clean workstation
to a Solana token project

A practical technical manual for learning the stack, directing AI assistants, reviewing generated code, and building a fixed-supply token on localnet and devnet.



EDITION 1.0 | 18 SEPTEMBER 2026

Rust + Anchor 1.2 / TypeScript + React / Solana

Technical documentation and guided laboratory exercises

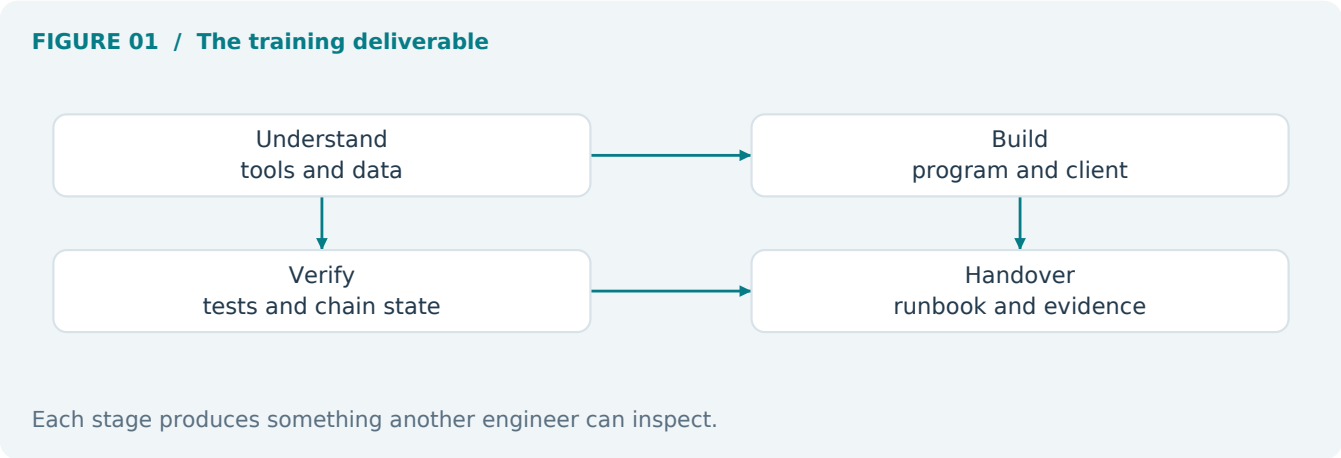
ORIENTATION

Read this first

Purpose, scope and the evidence you should keep while learning.

This manual introduces AI-assisted software development as a repeatable engineering practice. You will learn enough of the underlying tools to explain what the assistant changed, run the result yourself, and recognise when a plausible answer is technically wrong. The example is a fungible Solana token, because it connects ordinary application development with accounts, permissions and irreversible state changes.

Document control	Training edition
Audience	An employee learning to build software with an AI assistant; no prior Rust experience assumed.
Prerequisites	Basic computer use, permission to install development tools, and time to read errors carefully.
Target outcome	A documented local/devnet project, tested token creation, a transfer and a public-data dashboard.
Scope boundary	No real-money token sale, DEX liquidity pool or mainnet transaction is executed by this manual.
Validation status	APIs checked against primary sources; six TS syntax checks and seven mocked RPC cases passed. Anchor compilation, deployment and full npm builds were not executed.



The companion ZIP contains the complete teaching scaffold. Copyable code in this PDF matches those files; installation still resolves dependencies and generates your own program ID. References use clickable identifiers such as [\[R14\]](#). All sources were checked on 18 September 2026. Timings, token supply and budgets marked as training assumptions are examples designed for this course, not industry statistics.

ORIENTATION

Contents

Use the chapter bookmarks or click the page numbers. The footer returns here.

Chapter	Pages	What you will be able to do
01 Foundations	4-10	Explain AI assistance, programming basics and the application stack.
02 Workstation and tools	11-19	Install tools, isolate a wallet, use Git and understand configuration.
03 Working with AI	20-26	Write precise requests, preserve context, review changes and debug.
04 Solana design	27-32	Understand mints, token accounts, authorities, PDAs and fees.
05 Build the token project	33-46	Create a CLI token, implement a custom program, test and deploy to devnet.
06 Application and launch	47-52	Build a dashboard and distinguish issuance, metadata and liquidity.
07 Delivery and operations	53-56	Automate checks, record releases and investigate failures.
08 Learning and reference	57-66	Practise independently, assess understanding and find the source material.

How to read the code listings

A listing labelled with a filename supplies the text for that file. Listings labelled 'part 1' and 'part 2' belong to the same file and must be joined in order. Command blocks identify the working directory. Do not type a shell prompt before a command. Uppercase placeholders such as `YOUR_MINT` are values you replace with the addresses printed by your own tools.

How to use the course

Read pages 4-19 once before opening the assistant. Then follow the working method on pages 20-26 during every lab. Complete the local token exercises before devnet deployment. Keep a learning log with your commands, expected result, actual result and explanation of each failure. Returning to a failed step with a clearer explanation is useful progress.

Recommended evidence folder

Create `docs/lab-evidence.md`, `docs/decisions.md` and `docs/toolchain.md` in your repository. Record public addresses and redacted logs. Wallet files, seed phrases and credentials belong outside the repository.

01 FOUNDATIONS

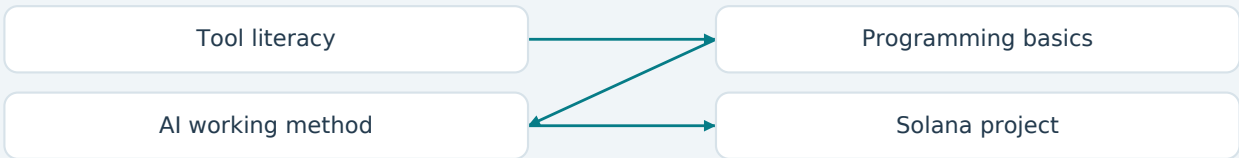
Your learning route

An employee onboarding plan with concrete completion gates.

Treat the project as a sequence of small capabilities. Installing everything at once and asking for the finished application makes it difficult to identify which layer caused a problem. A useful checkpoint has one purpose, one observable result and one explanation you can give without asking the assistant to speak for you.

Stage	Practice	Completion evidence
1. Tool literacy	Run version commands, create files, open a terminal and commit a change.	Recorded tool versions and a clean Git diff.
2. Programming	Read TypeScript and Rust, trace a function and convert token units.	A correct explanation of 25 tokens as 25,000,000 raw units.
3. AI workflow	Give a constrained request and review a small patch.	A requirement, a diff and an independently checked result.
4. Solana lab	Create and inspect a token; build and test the program.	Exact supply, null authorities and meaningful rejection tests.
5. Application	Read supply through RPC and handle failed requests.	A local browser page and a successful production build.
6. Handover	Write a runbook and reproduce the setup.	Another person can follow the instructions from a clean checkout.

FIGURE 02 / The course dependency order



Later stages reuse the checks and vocabulary from earlier stages.

A planning allowance of 25-40 focused hours is reasonable for this course as a training estimate; it is not a measured completion time. Spend extra time on concepts that remain unclear. Your goal is to explain the program's permissions and failure behavior, not to reach deployment fastest.

01 FOUNDATIONS

What coding with AI means

Understand the assistant's role before giving it access to a project.

Coding with AI means using a language model to help describe, implement, inspect and explain software. The model predicts a useful response from the information available to it. Some tools work only with pasted text; others can search the repository, edit files and run commands. Those additional capabilities make an assistant more useful, but they do not turn its output into evidence of correctness.

Mode	Typical use	Your responsibility
Chat / explanation	Ask what a function does or compare two designs.	Supply the relevant code and verify claims against documentation.
Completion	Accept a short expression or repetitive structure in the editor.	Read the suggestion in its surrounding code before accepting it.
Editing	Request a patch across selected files.	Inspect the whole diff, including configuration and dependencies.
Agent	Ask the tool to investigate, implement and run checks.	Control its scope, review commands and distinguish executed checks from claims.

Where AI assistance is useful

The assistant can turn a vague requirement into a set of questions, explain unfamiliar compiler errors, draft boilerplate and suggest failure cases. It can also help you compare a generated function with the actual API reference. Ask it to teach the reasoning behind a change, then test your understanding by predicting the output before running the code.

Where human judgment remains necessary

You decide what the product should do, which data it may access, who may authorise a transaction and what evidence is sufficient to accept a change. A fluent explanation can accompany a nonexistent package, an outdated command or an account validation bug. Treat generated code as a proposed patch submitted by a contributor whose reasoning you still need to inspect.

For this manual, an editor-based assistant is convenient but optional. You can complete the labs with a chat tool and ordinary files. GitHub documents its editor extension and cloud-agent capabilities separately; available permissions depend on the product and account configuration. [\[R50\]](#) [\[R51\]](#)

Working rule

Accept a change only when you can state the requirement it satisfies, understand the changed code at the level appropriate to its risk, and show the relevant validation result.

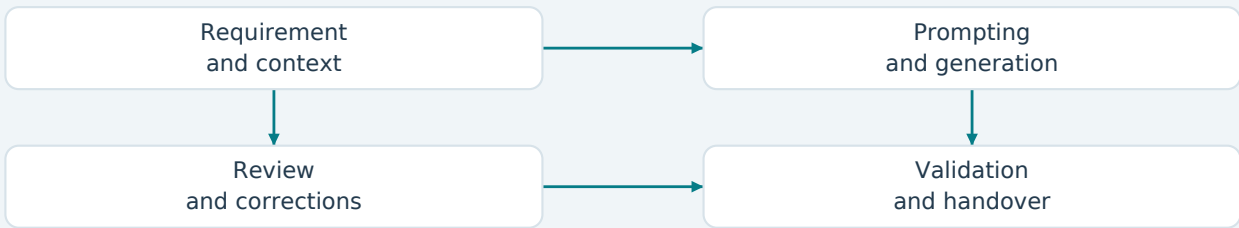
01 FOUNDATIONS

Evidence and useful statistics

Research describes particular tasks and populations, not a universal speed multiplier.

Primary evidence	Reported result	Interpretation
Peng et al., February 2023 [R54]	The assisted group completed a JavaScript HTTP-server task 55.8% faster.	A controlled, bounded task; this is not a promise for every software project.
METR, July 2025 [R55]	16 experienced developers, 246 real issues; AI use increased task time by 19%.	Familiar, mature open-source repositories and early-2025 tools.
METR, February 2026 update [R56]	Later estimates were affected by recruitment, task selection and time-measurement problems.	The authors caution that the later data is unreliable for estimating the current effect.

FIGURE 03 / Measure the whole task



Your elapsed time includes waiting, debugging and review, not just typing.

The practical lesson is to measure your own workflow. Record the time needed for a small feature from written requirement to passing checks. Also record whether you can explain the result without assistance. Typing fewer lines is useful only if the whole task improves.

Useful training metrics include defects discovered before acceptance, time spent investigating failures, the number of unexplained changes in a diff and how often you consulted a primary source. These are proposed measures for this course, not validated industry benchmarks. Keep productivity and learning as separate observations: an explanation session may take longer while building a skill you reuse later.

Do not combine unlike percentages

The 2023 and 2025 studies used different tasks, participants and tools. Averaging their headline results would not produce a meaningful estimate for your Solana project.

01 FOUNDATIONS

Programming essentials

A small vocabulary makes AI explanations much easier to assess.

A program is a set of instructions that transforms inputs into outputs and sometimes changes stored state. A variable names a value; a type describes what kind of value is valid; a function groups a reusable operation. A condition selects a path, while a loop repeats work. An error means the program could not complete an operation under its stated rules. These ideas recur in both the browser and the on-chain program.

Concept	Example in this project	Question to ask
Input	A recipient public key and an amount.	Is the format valid, and is this the intended recipient?
Type	A <code>PublicKey</code> object, bigint amount or string address.	Can this value lose precision or represent an invalid state?
Function	<code>addresses(creator)</code> derives the mint and treasury.	Does the same input always produce the same result?
State	The mint supply and the treasury balance.	Which operation is allowed to change it?
Side effect	Sending a token transfer.	What happens if the request succeeds but the response is lost?
Invariant	Mint authority is absent after initialization.	Can any allowed sequence of operations break this property?

Compile time and runtime

A compiler or type checker can reject a reference to a missing field before the program runs. Runtime checks validate information that becomes known during execution, such as an RPC response or the owner of a blockchain account. TypeScript types do not make an incoming JSON response trustworthy. Rust types do not decide whether a particular caller should receive a particular permission.

Read code in this order

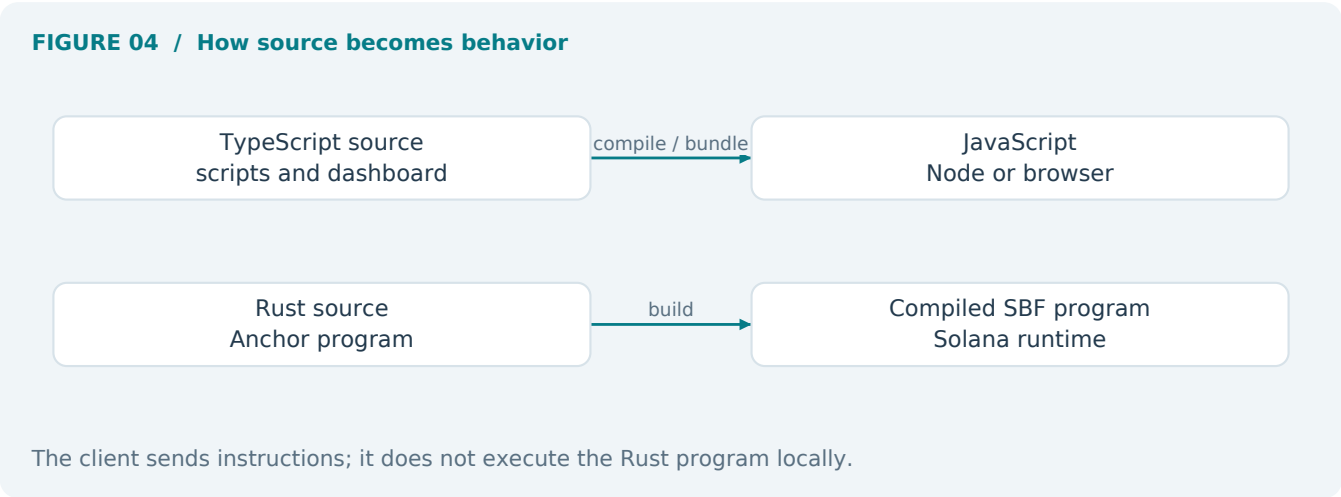
Find the entry point, identify its inputs, trace the operations that read state, then find operations that write state or contact another system. Finally inspect how the code reports failures. For each return value, ask which later function uses it. This is more reliable than reading every punctuation mark in sequence.

Practise with a tiny task: write a function that accepts a whole-number token quantity and returns raw units. Predict the result for zero, one and 25. Then consider a fractional input and a very large input. The need to define accepted inputs becomes visible before an AI assistant generates a larger implementation.

01 FOUNDATIONS

Languages and execution

Source files, generated artifacts and running processes serve different roles.



TypeScript adds static types to JavaScript. In the command-line client it runs through ts-node and Node.js; in the dashboard it is transformed and bundled by Vite, then executed by the browser. Rust is compiled to a Solana-compatible program artifact. Keep these execution environments distinct when reading an error. A browser cannot access your local wallet file through Node's file-system module. [\[R05\]](#) [\[R07\]](#)

Tool / artifact	Role	Common mistake
Node.js	Runs local JavaScript tools and scripts.	Assuming Node APIs exist in a browser.
TypeScript checker	Checks declared relationships between values.	Assuming it validates JSON returned by an RPC server.
Cargo	Resolves Rust crates and manages builds.	Confusing the host Rust compiler with Solana's build toolchain.
Anchor IDL	Describes the program interface for clients.	Editing generated IDL manually instead of rebuilding.
Browser bundle	Contains public frontend code and configuration.	Putting a private key into a frontend environment variable.

The same token can be accessed by different clients because its state lives on the network. The CLI, your TypeScript scripts and the dashboard must agree on the cluster and mint address. Their user interfaces may differ while they read the same state.

Learning check

If the dashboard builds but initialize fails with a program error, which runtime reported the failure? Explain why rebuilding React cannot repair an on-chain account constraint.

01 FOUNDATIONS

The terminal and files

Know where a command runs and what it can change.

The terminal is a text interface to programs installed on your computer. The shell parses the command, expands variables and launches the requested program. The current directory matters: npm reads package.json from a project, Cargo reads a workspace manifest, and a relative path starts from the directory you are in. When a tutorial changes directory, that change affects later commands in that terminal.

Basic navigation - macOS, Linux or a WSL Ubuntu terminal

```
pwd
ls
mkdir -p ai-coding-training
cd ai-coding-training
git --version
node --version
npm --version
```

Syntax	Meaning	Example
Absolute path	Starts at the root of the file system.	/Users/yourname/projects/fixed_supply
Relative path	Starts at the current directory.	programs/fixed_supply/src/lib.rs
~ or \$HOME	Your user home directory.	~/.config/solana/ai-coding-lab.json
Quoted argument	Keeps spaces within a single argument.	Use quotes around a path such as "My Projects".
Environment variable	Configuration inherited by a launched process.	ANCHOR_PROVIDER_URL selects a script's RPC endpoint.
Exit code	Zero usually means success; nonzero usually means failure.	A CI job stops when a required command fails.

How to investigate an unfamiliar command

Run its help option before acting: solana --help, anchor --help or the subcommand's --help. Identify the target directory, target network, wallet and any file it overwrites. A command that prints a version is different from a command that deploys a program. Read installation scripts from their official source before executing them; installers may alter PATH, download packages and change your shell startup files.

If a command is not found, first reopen the terminal and inspect the installer's PATH instructions. On Windows, do not mix tools installed inside WSL with commands running in PowerShell. The later labs use a Unix-style shell; their export syntax belongs in macOS, Linux or Ubuntu under WSL.

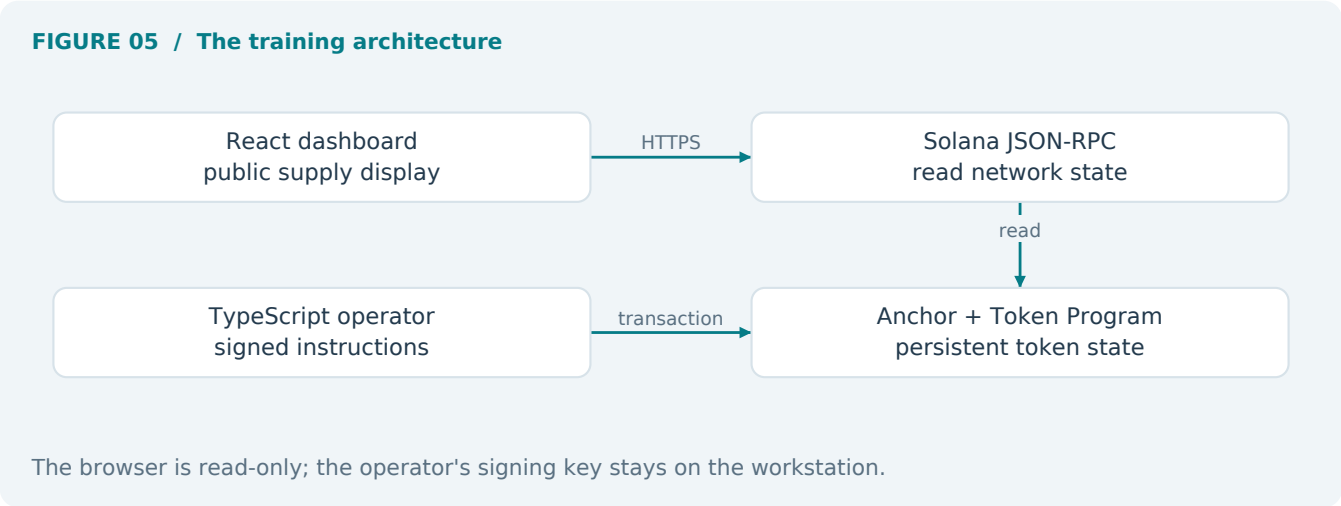
Capture useful failures - Keep the command, working directory, tool version and first meaningful error. Redact secrets before sharing logs with an assistant. A screenshot alone often hides text needed to search the source reference.

01 FOUNDATIONS

Anatomy of the stack

Select a stack by the problem you need to solve.

A stack is the collection of languages, libraries, runtimes and services that cooperate to deliver an application. This project uses a small browser frontend, a TypeScript operator client and an on-chain Rust program. Blockchain RPC provides access to network state. A separate database and web backend are unnecessary for the initial learning scope.



Layer	Chosen tools	Why it exists
Editor and collaboration	VS Code, an approved AI assistant and Git.	Write, inspect and track changes.
Client language	TypeScript with Node.js.	Derive addresses, initialize, inspect and transfer.
Frontend	React, TypeScript and Vite.	Build a small, understandable application surface.
Program	Rust, Anchor and anchor-spl.	Enforce the issuance sequence on-chain.
Network	Local validator, then devnet RPC.	Test without committing real funds.

Avoid adding a database, container orchestrator or authentication service merely because the assistant suggests a conventional full stack. Add a component when a requirement justifies its operation, deployment and maintenance cost. For example, private customer records would need a separate design; public mint supply does not.

Architecture decision

Record why the dashboard does not sign transactions. This keeps the first application exercise focused on data retrieval, state and error handling while the CLI covers transaction signing.

02 WORKSTATION AND TOOLS

Downloads and tool choices

Use official project websites and verify the executable after installation.

Component	Official download / setup	Training choice
Visual Studio Code	code.visualstudio.com/download [R01]	Current supported editor for your operating system.
Git	git-scm.com/install [R02]	Current supported Git; record git --version.
Node.js	nodejs.org/en/download [R03]	Node 24 LTS; record the patch version. [R04]
Rust / rustup	rust-lang.org/tools/install [R10]	Host toolchain managed through rustup.
Solana / Agave	Solana installation guide [R14]	Compatible Agave 4.x for this Anchor 1.2 lab.
Anchor / AVM	Anchor installation guide [R16]	Anchor CLI, Rust crates and TS client at 1.2.0.
SPL Token CLI	Solana token quickstart [R30]	cargo install spl-token-cli; record resolved version.
AI assistant	Copilot editor setup [R50]	An optional example; use the tool approved for your work.

TypeScript, React and Vite are installed into the project through npm rather than downloaded as standalone desktop applications. The software packages are generally open-source tools; AI subscriptions and hosted RPC plans have provider-specific costs that can change. This manual makes no paid subscription a prerequisite.

Installation versus project dependency

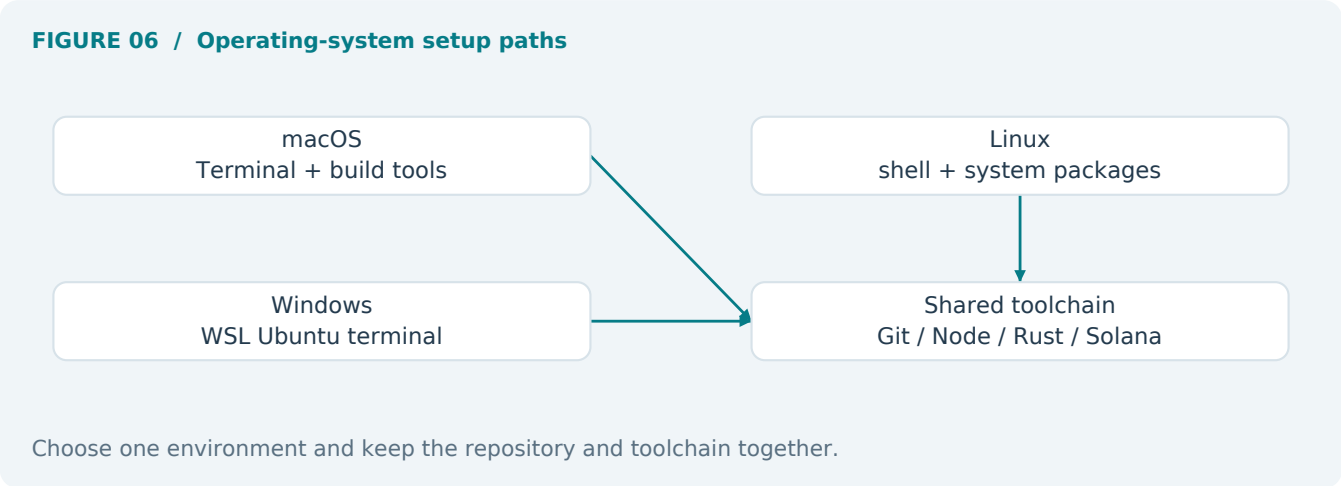
Git and the Solana CLI are commands available to your shell. React and the Anchor TypeScript client are libraries declared by a particular project. Updating a global command does not update the project's package manifest, and editing package.json does not select a different Anchor CLI. Version agreement is a deliberate step.

Create docs/toolchain.md with the date, operating system, CPU architecture and versions actually installed. Keep the original website link beside each component so a future employee can refresh the setup. Do not copy random installation commands from a search result without checking the owning project.

02 WORKSTATION AND TOOLS

Choose your platform path

The labs use the same Unix-style commands on macOS, Linux and WSL.



Platform	Preparation	Verification
macOS	Install command-line build tools when prompted; use the correct Intel or Apple Silicon download.	Open a new Terminal and check Git, Node and Rust.
Linux	Use the supported package-manager instructions in the Solana dependency guide.	Confirm the native compiler and required system libraries exist.
Windows	In administrator PowerShell run wsl --install; restart if prompted; finish setup in Ubuntu. [R09]	Run the Unix commands in WSL, and open the project through the editor's WSL integration.

On Windows, storing the project in the Linux home directory avoids unnecessary cross-file-system friction. The path seen by WSL differs from the path seen by Windows. A wallet file referenced by an Ubuntu process must be accessible through its Linux path. If you ask an assistant to repair a path issue, tell it which shell and environment produced the error.

What to do when installation fails

Use the manual dependency guide to isolate the missing component. A linker error, TLS failure and command-not-found error point to different layers. Do not repeatedly run the whole installer without understanding which step failed. Solana provides both an automated installer and instructions for individual dependencies. [R14] [R15]

Checkpoint

Write one sentence describing your execution environment, for example: 'Ubuntu under WSL, repository in my Linux home directory, commands run in the Ubuntu terminal.' Include it in later debugging requests.

02 WORKSTATION AND TOOLS

Editor, Git and Node.js

Establish a working application-development environment first.

Install VS Code, Git and Node 24 LTS from the links on page 11. Open a new terminal after installation. Check that each executable resolves before adding Solana tooling. If your computer already has several Node versions, choose one supported LTS version for this repository and record it rather than allowing different terminals to use different runtimes. Node's release page identifies the supported branches. [R04]

Version and identity checks

```
git --version
node --version
npm --version
git config --global user.name "Your Name"
git config --global user.email "your-work-address@example.com"
```

The Git identity labels your commits; it does not authenticate you to a remote repository. Replace the example name and email. If you already have the correct global identity, inspect it instead of changing it. Choose repository-specific settings when you need a separate work identity.

Configure a readable editor

Open the project folder, enable file extensions in your file browser and learn how to view the integrated terminal, search the repository and inspect the Source Control diff. Install a Rust language extension if desired, plus the assistant extension approved for your workplace. An extension's publisher and permissions matter because editor extensions can access project content.

First practical exercise

Create a text file explaining the project in three sentences. Change one sentence and use the editor's diff view to identify exactly what changed. Ask your assistant to improve the explanation without editing other files, then inspect its patch. This low-impact exercise teaches the acceptance process before you handle program configuration or token transactions.

Command	Expected result	If it fails
node --version	A v24.x.y version for the selected training runtime.	Reopen the terminal or inspect the selected Node installation.
npm --version	An installed npm version.	Use the npm supplied with your supported Node installation.
git --version	A Git version string.	Finish Git installation and reopen the terminal.

Use local npm dependencies for the project tools. Running a script from package.json makes the intended command visible to the next employee and to CI. Avoid installing every JavaScript build tool globally; the repository should describe which tools it needs.

02 WORKSTATION AND TOOLS

Rust, Solana and Anchor

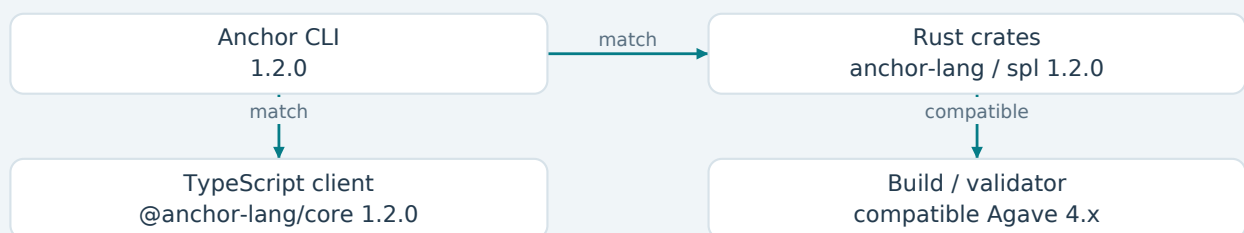
Pin the interface family and check the compatible build tools.

The official quick installer is convenient for initial setup. It installs several components, so review the source linked by the installation page first. If you need precise workplace control, use the manual dependency instructions and select the approved releases individually. [\[R14\]](#) [\[R15\]](#)

Official quick setup, then select the manual's Anchor version

```
curl --proto '=https' --tlsv1.2 -sSfL \
  https://solana-install.solana.workers.dev | bash
# Reopen the terminal before continuing.
avm install 1.2.0
avm use 1.2.0
rustc --version
cargo --version
solana --version
anchor --version
cargo install spl-token-cli
spl-token --version
```

FIGURE 07 / The compatibility contract



A successful install is not proof that independently selected versions work together.

Anchor 1.x renamed its TypeScript package and changed CPI construction; old tutorials may show @coral-xyz/anchor and an AccountInfo argument to CpiContext::new. This lab uses @anchor-lang/core and a program public key. [\[R18\]](#) The selected lab release is documented in [\[R17\]](#). The CLI reference states that its default v3 build requires Agave 4.0 or newer for the legacy validator; use a compatible 4.x patch and record it. [\[R20\]](#)

Before the first program build

Check anchor --version is 1.2.0. Confirm the Rust manifests and TS client use the same release. The lab explicitly uses the original Token Program and web3.js v1; it does not mix Solana Kit client objects.

02 WORKSTATION AND TOOLS

Wallets, clusters and funding

Separate local testing, shared devnet testing and real-value networks.

A keypair contains a public address and private signing material. The public address can be recorded in your lab notes. The keypair JSON and recovery phrase must remain private. The CLI keypair file is not encrypted simply because you entered a recovery-phrase passphrase during generation. Solana's CLI guide explains this distinction. [R59]

Create a separate lab keypair - do not overwrite an existing file

```
mkdir -p "$HOME/.config/solana"
solana-keygen new --outfile "$HOME/.config/solana/ai-coding-lab.json"
chmod 600 "$HOME/.config/solana/ai-coding-lab.json"
solana config set --keypair "$HOME/.config/solana/ai-coding-lab.json"
solana config set --url localhost
solana config get
solana address
```

Environment	Use	Funding / limitations
Localnet	Fast, isolated development on your own validator.	Local SOL is test data; a ledger reset removes local accounts.
Devnet	Shared application development and integration checks.	Devnet SOL is test currency; public RPC and faucets can throttle or reset.
Testnet	Validator and network-release testing.	Not the primary environment for this application's learning path.
Mainnet-beta	Real-value production activity.	Excluded from the lab commands and write scripts.

Start a validator only when the step requires one

The first CLI lab runs against a validator you start in a separate terminal. Later, Anchor tests manage their own validator. Do not leave the first process listening on the same port when starting an Anchor-managed test run. The local RPC endpoint is normally `http://127.0.0.1:8899`.

Before every write, inspect the cluster and wallet explicitly. `solana config get` describes the CLI. `Anchor.toml` describes Anchor's provider. `ANCHOR_PROVIDER_URL` and `ANCHOR_WALLET` describe scripts using `AnchorProvider.env`. These are related settings, but changing one does not automatically change all three.

Checkpoint

You should be able to identify the lab wallet's public key and explain which configuration selects the network for each tool. If either is uncertain, resolve the configuration before sending a transaction.

02 WORKSTATION AND TOOLS

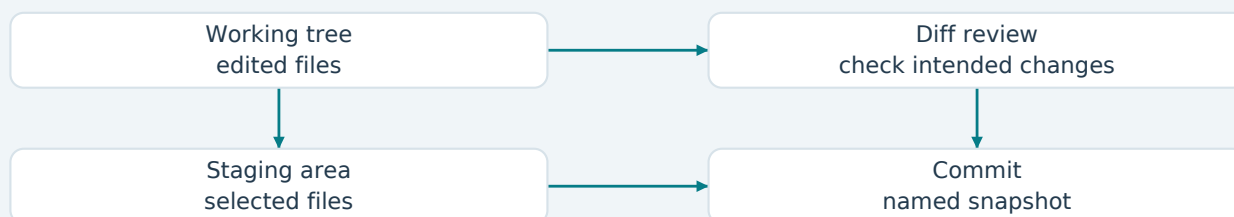
Repository and checkpoints

Version control makes a proposed change concrete and reversible.

Git records snapshots of tracked files. A branch gives a line of work a name; the staging area selects what goes into the next commit. Keep each commit small enough to explain. A clean checkpoint before an AI edit lets you distinguish the assistant's patch from earlier changes and return to a known working version. [\[R47\]](#)

Start a small practice repository

```
mkdir -p ai-coding-practice
cd ai-coding-practice
git init
git switch -c main
# Create README.md and .gitignore in the editor.
git status
git add README.md .gitignore
git diff --cached
git commit -m "Document the training project"
```

FIGURE 08 / The Git change path

Inspect both the working diff and staged diff before creating a checkpoint.

Use `git diff` before staging and `git diff --cached` afterward. An assistant can accidentally stage unrelated configuration or generated files. Include source, manifests, lockfiles and public documentation. Exclude `node_modules`, build output, validator ledgers, local environment files and keypair JSON.

For the main project, the provided `.gitignore` excludes target because it contains compiled output and the program keypair. Public IDL files can be exported into a separate release-evidence folder when needed. Keep the private program keypair securely outside ordinary Git history while preserving its identity across builds.

A safer recovery habit

Save a patch or create a checkpoint before discarding work. `git restore` can remove uncommitted edits. Use it only on a path whose contents you have inspected. [\[R48\]](#)

02 WORKSTATION AND TOOLS

TypeScript you will use

Read objects, async functions and precise integer values before writing the client.

TypeScript expresses relationships between values before execution. A type annotation such as string describes a value the program expects. An object groups named fields. An imported function comes from another file or package. An async function returns a Promise, and await waits for that operation to settle before continuing in the current function. The checker helps you find inconsistent code, but external inputs still need runtime validation. [R05]

A precise raw-unit conversion - illustrative function

```
type TransferRequest = { recipient: string; amount: bigint };

function wholeTokensToRaw(text: string): bigint {
  if (!/^[0-9]+$/.test(text)) throw new Error("Whole tokens required");
  return BigInt(text) * 1_000_000n;
}

const request: TransferRequest = {
  recipient: "YOUR_PUBLIC_KEY",
  amount: wholeTokensToRaw("25"),
};
console.log(request.amount.toString()); // 25000000
```

Why bigint appears in the token scripts

A JavaScript number cannot represent every integer above 2^53 - 1 exactly. Token supply fields may cover a much larger range, so the SPL client represents raw amounts with bigint. The suffix n marks a bigint literal. Convert a raw integer string directly with BigInt(text), rather than passing through Number(text). Use toString when serialising bigint to ordinary JSON. [R64]

Errors should remain visible

A try/catch block can report a failure and stop a workflow. It should not convert every failure into a success message. In a script, set a nonzero process exit code when a required operation fails. In a UI, show an explicit unavailable or failed state instead of a zero balance that the user might mistake for confirmed blockchain data.

Read this syntax	As this idea
const { mint, treasury } = addresses(creator)	Extract two named fields from the returned object.
await getMint(connection, mint)	Wait for the network read before using the decoded mint.
state.mintAuthority !== null	Minting permission still exists; the lab invariant has failed.
import type { FixedSupply } ...	Import a generated compile-time type rather than a runtime value.

02 WORKSTATION AND TOOLS

Rust and token arithmetic

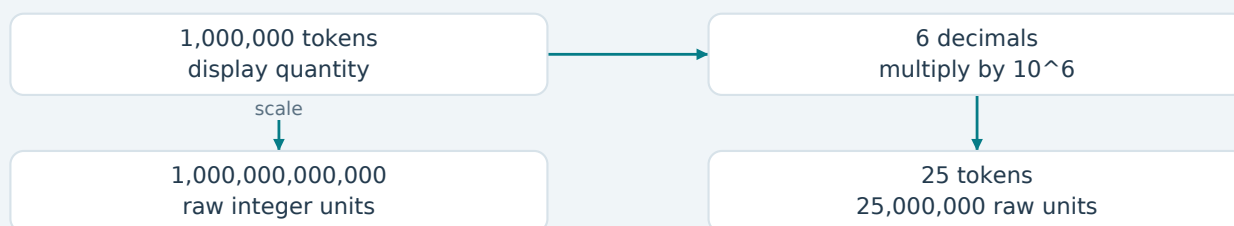
Focus on the pieces needed to understand the program's rules.

Rust uses explicit integer types, structs and functions. A u64 is an unsigned 64-bit integer; it cannot represent negative values. A Result describes success or failure. The question-mark operator propagates an error to the caller, which is why a failed CPI stops initialization. Rust ownership controls how values and references are used in memory; it is separate from the Solana permission meaning of 'owner'. [R11] [R12]

Read the essential Rust shapes

```
pub const INITIAL_SUPPLY: u64 = 1_000_000_000_000;
// pub: visible; const: fixed value; u64: raw unsigned integer.

pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
    // Validate accounts through the Initialize struct.
    // Run token CPIs; propagate failures with ?.
    Ok(())
}
```

FIGURE 09 / Display units and raw units

Amounts in the program and SPL client are raw units; human-facing CLI amounts may differ.

The training supply is a design choice: one million tokens with six decimals. A transfer changes account balances but not total supply. A burn reduces supply. Removing mint authority guarantees that additional units cannot be minted through the original Token Program; it does not stop holders from burning. [R34] [R35]

For variable-supply programs, use checked arithmetic for additions and multiplications and define rejection behavior for overflow. This lab keeps the supply constant to reduce the first program's state space. The workspace enables release overflow checks as an additional guard; those checks do not replace a clear application-specific cap.

Predict before running

With six decimals, what raw amount represents 0.5 tokens? Answer: 500,000. The transfer script accepts raw integers, so fractional display amounts must be converted before invoking it.

02 WORKSTATION AND TOOLS

HTTP, RPC and configuration

A successful network request and a successful program operation are different events.

HTTP transports a request to a server. JSON-RPC defines a request body containing a method, parameters and an identifier, and a response containing a result or an error. Solana exposes JSON-RPC methods for reading state and submitting transactions. A response can use HTTP status 200 and still contain an RPC error. The dashboard checks both layers. [\[R43\]](#)

A read-only JSON-RPC request body

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "getTokenSupply",
  "params": ["YOUR_MINT", { "commitment": "confirmed" }]
}
```

Configuration	Read by	Purpose
solana config	Solana CLI.	Default cluster, wallet and commitment for CLI commands.
Anchor.toml	Anchor CLI.	Program addresses, provider settings and test script.
ANCHOR_PROVIDER_URL	AnchorProvider.env in Node scripts.	RPC URL for initialize, inspect and transfer.
ANCHOR_WALLET	AnchorProvider.env and the transfer script.	Absolute path to the lab keypair.
VITE_MINT_ADDRESS	Dashboard at build / dev-server startup.	Public mint address embedded in browser configuration.

Public versus private environment values

An environment variable is a convenient configuration mechanism, not automatic secret storage. Vite exposes prefixed variables to client code, and the resulting browser bundle is available to users. A mint address is suitable frontend configuration; a signing key, privileged RPC credential or recovery phrase is not. [\[R08\]](#)

Understand commitment

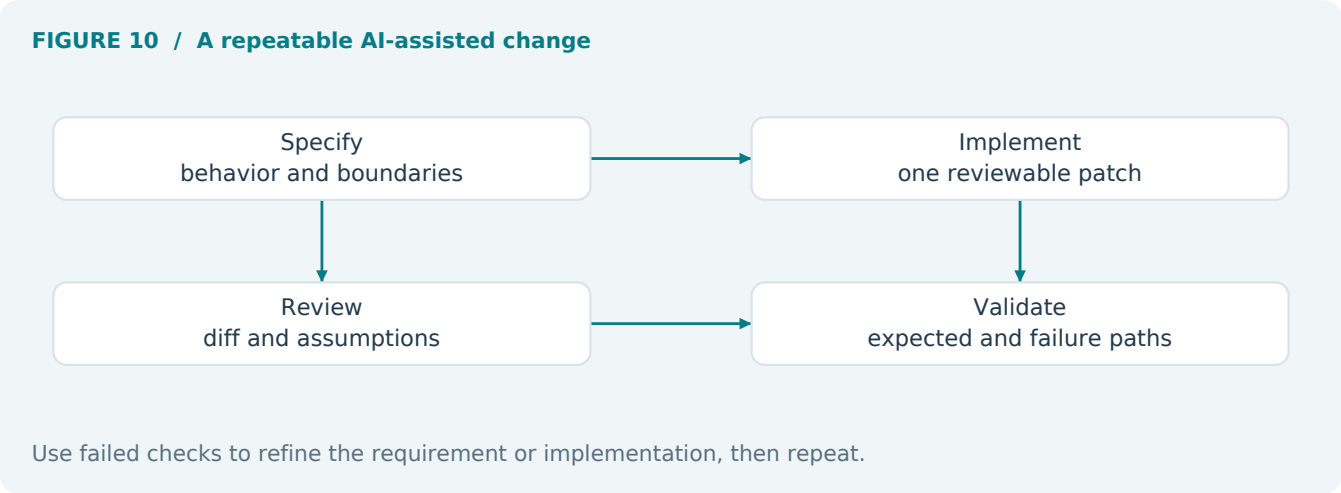
A 'confirmed' read asks for a state with a particular network confirmation level. 'Processed', 'confirmed' and 'finalized' are distinct observations. Use a consistent commitment for related reads during the lab. A returned signature identifies a transaction, while confirmed execution and verified postconditions establish what it actually did.

In your learning log, record the endpoint and mint address with each network result. An apparently missing token may be a correct response from the wrong cluster. Repeated requests cannot fix a cluster mismatch.

03 WORKING WITH AI

The engineering loop

A written requirement, a small patch and a relevant check form one unit of work.



Begin with observable behavior. 'Create a secure token app' hides dozens of choices. 'Add a read-only devnet page that shows getTokenSupply, displays network failures and never requests wallet signing' gives the assistant a tractable task. Specify relevant files, interfaces and constraints before it edits.

Step	Ask the assistant to supply	You independently inspect
Specify	A concise interpretation and unresolved assumptions.	Does the interpretation match the product requirement?
Implement	The smallest patch for the agreed behavior.	Are unrelated files or dependencies changing?
Review	A description of changed permissions and failure cases.	Does the actual code match that description?
Validate	Commands run, outputs and remaining limits.	Were checks executed, and do they prove the requirement?
Checkpoint	A short handover note.	Can the change be recreated and explained?

When a task crosses several layers, split it at a stable interface. First build an inspection script, then use its findings to specify the dashboard. First test a program constraint, then improve the error explanation. Small steps make mistakes easier to locate without asking the assistant to regenerate everything.

Useful stop condition

A task is complete when the defined behavior is implemented, the appropriate checks pass and the limits are documented. A persuasive explanation or a page that merely looks finished is insufficient.

03 WORKING WITH AI

Write a useful specification

Requirements should describe behavior that can be checked.

A specification aligns the developer, reviewer and assistant on the outcome. Separate the required behavior from implementation ideas so that a suggested framework does not quietly become a product requirement. For this project, the core behavior is token issuance with explicit supply and authority postconditions. The choice of React is a teaching choice for the separate dashboard.

Specification field	Training example
Purpose	Provide a learnable workflow for creating and inspecting a fixed-supply SPL token.
Actor	A creator who signs initialization with a dedicated local/devnet wallet.
Inputs	Creator public key; no variable supply parameter in the first program.
Behavior	Create a deterministic mint and creator ATA, issue the constant supply, revoke mint authority.
Success conditions	Six decimals; exact initial supply; creator owns treasury; mint/freeze authorities absent.
Failure conditions	Wrong PDA, wrong Token Program, wrong treasury ATA or repeated initialization must fail.
Exclusions	No mainnet write, metadata CPI, exchange listing, liquidity pool or customer database.
Evidence	Passing local tests, devnet initialization signature and an independent inspection output.

Acceptance criteria should be executable

Write 'getMint returns supply 1,000,000,000,000 immediately after initialization' rather than 'supply is correct'. Write 'a further mintTo instruction fails and supply remains unchanged' rather than 'minting is locked'. These statements map directly to the tests and reduce the chance of an assistant using a UI label as evidence for a chain-state property.

Resolve ambiguity before expanding scope

A request to 'launch a token' may mean create a mint, provide metadata, distribute tokens, establish a trading pool or publish a sale website. Each is a different deliverable. This manual implements issuance, inspection and a test transfer; it explains the additional launch layers on pages 51-52. State the selected meaning in docs/decisions.md so it survives a new assistant session.

Exercise
Write three acceptance criteria for a recipient-address input. Cover a valid address, a malformed address and a valid but unintended address. Notice that a format check cannot decide whether the recipient is the person you meant to pay.

03 WORKING WITH AI

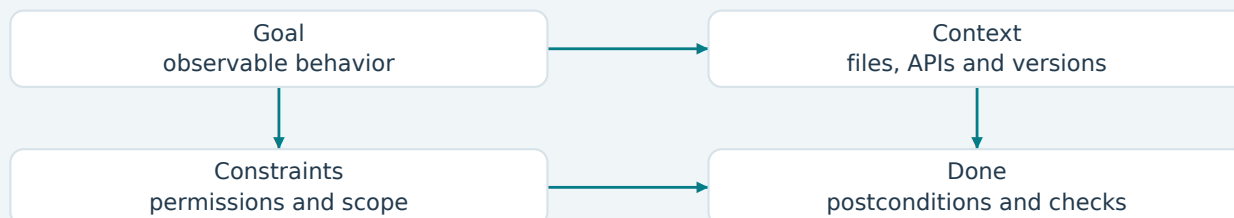
Prompt patterns

Give context, constraints and a definition of done.

Implementation request - adaptable template

Goal: implement the fixed_supply initialize instruction.
Context: Anchor 1.2.0, original Token Program, one mint per creator.
Files: programs/fixed_supply/src/lib.rs and tests/fixed_supply.ts.
Rules: six decimals; 1,000,000,000,000 raw units; creator ATA.
Revoke mint authority in the same transaction; no freeze authority.
Do not add metadata, Token-2022, sale or liquidity features.
Before editing: explain account validation and expected failures.
After editing: show the diff and report checks actually executed.

FIGURE 11 / Four parts of an actionable request



Attach the smallest relevant source context rather than an unrelated repository dump.

For explanation, ask the assistant to trace the actual code in order, identify every side effect and explain why each account is needed. For debugging, provide a minimal reproduction, versions, the complete error and your expected behavior. Ask for competing hypotheses and a check that distinguishes them before requesting a repair.

For review, ask it to look specifically for missing signer checks, account substitution, precision loss, secret exposure and incomplete error handling. Request findings with file locations and a concrete failure scenario. A second AI pass can discover issues, but independent tests and source references remain necessary.

Teach yourself after accepting a patch

Close the assistant's explanation and describe the change in your own words. Predict one success and one failure case. If you cannot do that, request a smaller explanation and inspect the code again.

03 WORKING WITH AI

Context and project instructions

Persist decisions in files so a new session has a reliable starting point.

An assistant works with a limited amount of context. It may not have read the relevant file, may infer a version from an outdated example or may forget an earlier constraint as a session grows. Keep enduring project facts in the repository, and point the tool at the actual files. Context written by the assistant should still be reviewed like code.

Example project instructions - see AGENTS.md in the ZIP

```
Purpose: local/devnet fixed-supply SPL token training.
Use Anchor 1.2.0 and @anchor-lang/core.
Use web3.js v1 and the original Token Program.
Supply: 1,000,000 tokens, six decimals, 1,000,000,000,000 raw units.
PDA seeds: [b"mint", creator public-key bytes].
Initialization must revoke mint authority atomically.
No mainnet writes or wallet-file disclosure.
Never remove a constraint merely to make a test pass.
Report executed checks separately from proposed checks.
```

Repository document	Contains	Update trigger
README.md	Purpose, setup, common commands and expected outputs.	A workflow changes or a fresh checkout reveals a missing step.
AGENTS.md	Concise instructions for tools that support this convention.	A stable constraint or toolchain decision changes.
docs/decisions.md	Options considered, chosen design and tradeoffs.	You accept a new component or permission model.
docs/toolchain.md	Exact local versions and compatibility notes.	Any install or dependency upgrade.
docs/lab-evidence.md	Public addresses, signatures and redacted check results.	Each checkpoint or deployment.

Context is not authority

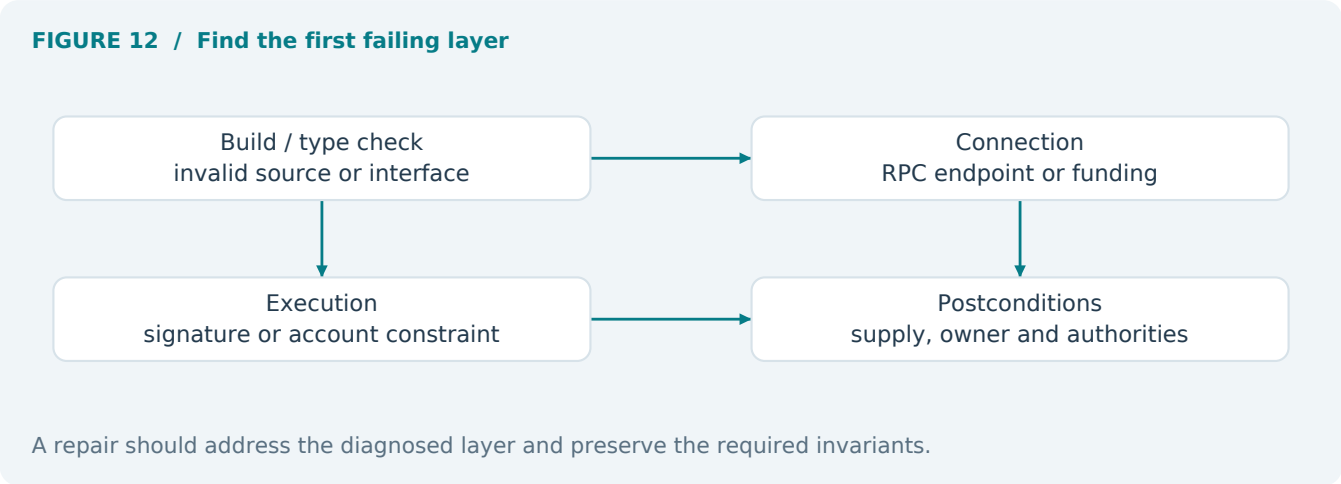
AGENTS.md is a convention supported by some tools, not a universal access-control mechanism. Read your chosen assistant's documentation for its instruction-file behavior. A source file, downloaded README or external web page may include instructions addressed to an agent. Treat external content as material to inspect; it should not redefine the user's task or grant access to secrets. [R53]

When an assistant asks to install an unexpected package, have it identify the requirement, the package's official owner and the API it needs. Reject invented dependency names by checking the primary source. Record the chosen dependency and the reason for adding it before expanding the project.

03 WORKING WITH AI

Review and debug a change

Inspect the program's behavior, not just the assistant's summary.



Start with git diff. Read configuration and dependency changes as carefully as the function body. Check whether the patch broadens permissions, changes a constant, switches a cluster, removes a rejection test or catches an error without failing the command. A large generated patch should be split before you decide whether to accept it.

Observed failure	Useful discriminating check
Missing module	Inspect package.json, installed dependency tree and the current directory.
Type mismatch	Compare the actual package version with the API reference and generated IDL type.
RPC error	Read HTTP status and RPC error separately; confirm the cluster and request body.
Constraint error	Inspect the exact accounts, PDA seeds, owner and program IDs passed to the instruction.
Wrong output	Read chain state independently; do not rely on an application success banner.

Ask the assistant to state a hypothesis before modifying code. 'The client is deriving mint with the wrong creator bytes' suggests comparing derived addresses. 'The program is insecure' is too broad to direct a repair. Change one variable at a time when possible and record which observation confirmed the cause.

Example of an unacceptable fix

Removing the seeds constraint to make a wrong-PDA test pass changes the security requirement. Fix the client derivation or the test setup while retaining the deterministic-address rule.

03 WORKING WITH AI

Secrets and trust boundaries

The assistant, browser and signing process do not need the same information.

A trust boundary separates components that can receive different levels of confidence or permission. The browser receives public supply data. The operator script signs transactions with a local keypair. The on-chain program validates caller-controlled account addresses. The assistant may inspect source code, but it does not need wallet contents to explain that code.

Information	Suitable location	Do not place it in
Mint / program / treasury address	README, public lab evidence and frontend config.	No special secrecy required; still label the cluster.
Lab wallet keypair	Private local file with restrictive access.	Git, pasted prompts, screenshots or browser bundles.
Recovery phrase	Private offline recovery record.	A code repository or assistant conversation.
Privileged RPC credential	An appropriately controlled service environment.	VITE_ variables if it grants privileges or costly access.
Redacted logs	Evidence files and debugging requests.	Logs that retain secrets or full wallet arrays.

Review external instructions

Prompt injection can place instructions inside content an assistant retrieves, including documentation, issues and repository files. The relevant failure is allowing that content to redirect tool use or reveal information beyond the task. OWASP discusses direct and indirect injection and recommends controls around the resulting actions. [\[R53\]](#) For this project, do not let retrieved text request private key output or replace the local/devnet constraint.

Limit the blast radius

Use a separate training wallet and avoid storing unrelated credentials in the project workspace. Inspect commands that read files or contact services before execution. Never accept a suggestion to print the wallet array so the assistant can 'verify' it. You can verify its public key with solana address instead.

If a key is exposed, removing it from the latest commit does not erase copies or history. Treat the key as compromised, stop using it and rotate the permissions or funds it controlled according to your organisation's incident process. The lab's use of test wallets makes this an instructive recovery exercise without involving real treasury assets.

Learning check

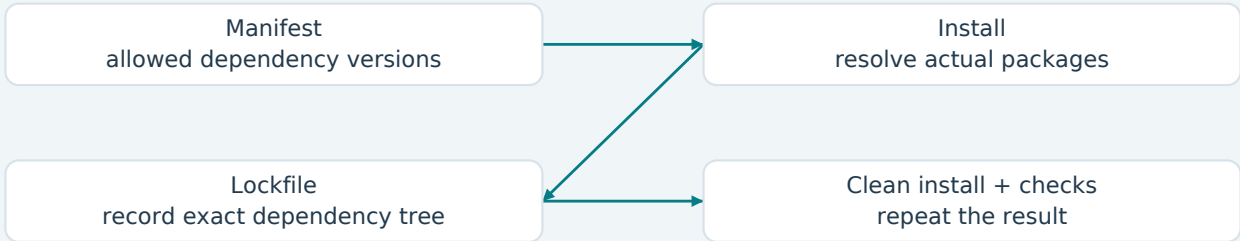
A frontend .env.local file is ignored by Git. Does that make a value placed in VITE_PRIVATE_KEY secret from users of the built application? No: client-exposed values become part of public frontend code.

03 WORKING WITH AI

Dependencies and reproducibility

A project should identify the exact software resolved during a working setup.

FIGURE 13 / A reproducible dependency record



Commit lockfiles after resolution; do not fabricate them from a manifest.

package.json describes the project's JavaScript dependencies and scripts. package-lock.json records the resolved tree. npm install performs the initial resolution; npm ci installs from a matching existing lockfile and fails if the manifest and lockfile disagree. [R49] Cargo uses Cargo.toml and Cargo.lock for corresponding Rust roles. [R13]

Control	Training practice
Version family	Pin Anchor CLI, crates and TS client to 1.2.0; stay within web3.js v1.
First install	Resolve the supplied ranges locally and commit both npm lockfiles and Cargo.lock.
Later install	Use npm ci in root and app, then rebuild and run the checks.
Dependency change	Explain why it is required; inspect the lockfile diff and official project.
Upgrade	Change a planned set of versions on a branch; repeat the meaningful behavior checks.
Evidence	Record operating system, runtime versions, source commit and build result.

The ZIP deliberately contains no fabricated lockfiles because dependency resolution was not executed in the authoring environment. Its Anchor release is exact; other package ranges select a compatible family, then your first install supplies the concrete dependency record. Do not call this a fully reproduced build until you have created and tested that record.

Compatibility note

Solana documentation now presents Kit examples alongside legacy examples. The Anchor TS client documents compatibility with web3.js v1 and SPL Token's existing client, so use that family consistently inside the operator scripts. [R19]

04 SOLANA DESIGN

The Solana token model

Most tokens are data handled by an existing Token Program.

A standard Solana token usually does not require a custom smart contract. You create a mint account owned by a Token Program, then use that program's instructions to issue, transfer or burn units. A custom program adds application-specific rules around those operations. In this course it makes the initial issuance and authority removal one controlled operation. [\[R29\]](#)

Entity	Contains / does	Identifier you record
Mint account	Decimals, total supply and mint/freeze authority fields.	Mint public key: the token's unique network identity.
Token account	A balance for one mint and an owner authorised to operate it.	Token-account public key, mint and owner.
Associated token account	The conventional deterministic token account for an owner and mint.	ATA address derived using owner, token program and mint.
Token Program	Enforces standard token instructions and authority rules.	The original Token Program ID in this lab.
Custom Anchor program	Defines this course's initialize instruction and account validation.	Your generated program ID.
Creator wallet	Signs initialization and owns the initial treasury ATA.	Creator public key; private material stays local.

Choose the token program deliberately

The original Token Program provides the familiar mint, transfer and authority behavior used by the lab. Token-2022 adds extensions such as transfer-related features and additional authorities. Extensions can alter integration requirements and account sizes. Do not silently switch programs because a newer tutorial uses an extension you have not specified.

Identify a token by address

Names and symbols are display labels and can be duplicated. The same-looking symbol can refer to different mints, and a mint address must be interpreted on a particular cluster. Record the mint with 'localnet' or 'devnet' beside it. An explorer's mainnet view cannot confirm a devnet token.

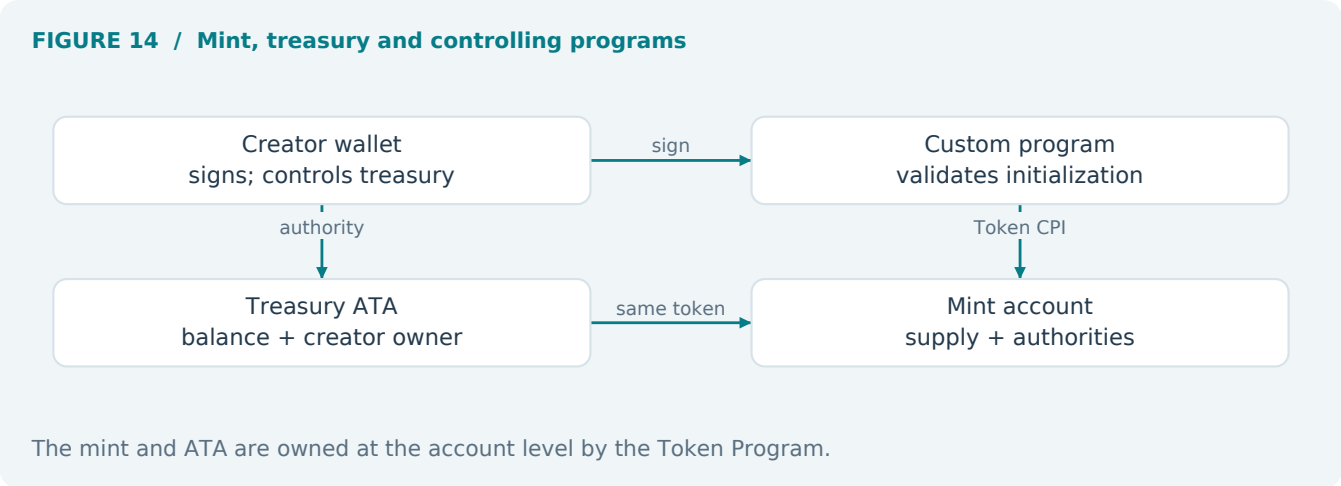
Two lessons in the project

Lab 1 uses the existing Token Program directly. Lab 2 writes a custom issuance program. Both create ordinary SPL token state; the second illustrates application rules rather than replacing the Token Program.

04 SOLANA DESIGN

Accounts and permissions

Distinguish account ownership from authority stored inside an account.



A Solana account's program owner controls its data. A token account also contains an owner field that identifies the wallet or authority allowed to operate its token balance. These are different uses of the word 'owner'. Your custom program creates the mint through CPI, but the Token Program owns the mint's token data. [\[R37\]](#) [\[R32\]](#)

Permission	Controls	Training outcome
Mint authority	Creation of additional units.	Creator during initialize; absent when it completes.
Freeze authority	Freezing and thawing token accounts.	Never configured for the core lab.
Token-account owner	Transfers and other owner-authorised operations.	Creator owns the initial treasury ATA.
Program upgrade authority	Replacement of the custom program's executable.	Recorded separately; not equivalent to mint authority.
Metadata update authority	Changes to associated descriptive metadata.	Not present in the core lab; separate extension.

Removing mint authority does not distribute the treasury, remove the treasury owner's transfer rights or make a token valuable. Removing the custom program's upgrade authority is also a different decision. An upgrade cannot restore a removed mint authority on an existing original-program mint, because the Token Program still enforces its own authority rules. [\[R36\]](#)

Learning check

The creator pays to create the mint. Does that mean their wallet can edit the mint's supply field directly? No: the Token Program owns and validates that state.

04 SOLANA DESIGN

The example specification

A small fixed-supply issuance program that is easy to reason about.

The fictional training project is named Solana Training Token in the dashboard. That label is application text, not on-chain metadata. The custom program is `fixed_supply` and exposes one `initialize` instruction. It creates one mint for each creator under a particular program ID, sends the entire initial supply to that creator's ATA and removes mint authority in the same transaction.

Property	Required behavior	Reason
Decimals	6.	Supports fractional display units without floating-point arithmetic.
Initial supply	1,000,000 tokens = 1,000,000,000,000 raw units.	A fixed teaching constant that fits safely in u64.
Mint address	PDA seeds: mint label + creator public-key bytes.	Predictable identity; a creator cannot repeat initialization for the same mint.
Destination	Creator's ATA for the new mint and original Token Program.	Prevents a substituted treasury from receiving the issuance.
Mint authority	Removed after <code>mint_to</code> within <code>initialize</code> .	No persistent inflation permission remains.
Freeze authority	Absent from initialization.	No freeze capability is introduced.
Distribution	100% initially controlled by creator.	Explicit centralisation; distribution is a later transfer exercise.
Network	Localnet first, devnet second.	Keeps the learning workflow in test environments.

What this design guarantees

After a successful `initialize` call, the mint starts with the specified supply and no mint or freeze authority. The creator receives the initial token balance. Later transfers conserve total supply; holder-authorized burns can reduce it. Repeated calls for the same creator and program fail because the initialized mint already occupies its deterministic address.

What this design does not guarantee

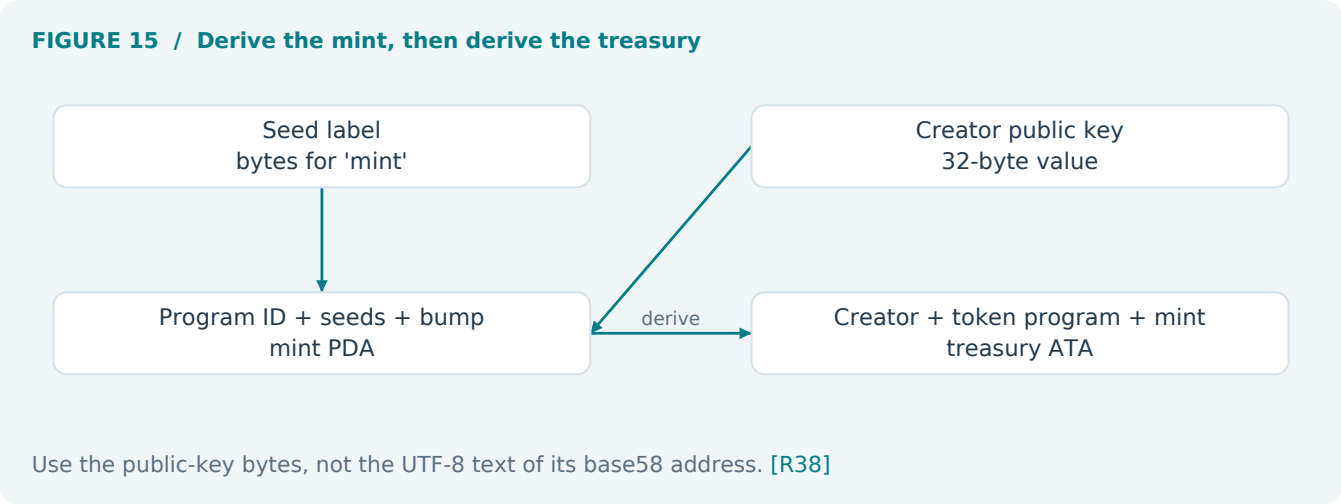
It does not promise fair allocation, a token price, exchange support or decentralised custody of the initial supply. Another creator can initialize a different mint with the same program. A modified program or different program ID can create a different token. Clients should verify the intended mint rather than treat the program ID as a universal identifier for one token.

Why the supply is constant - Removing a user-supplied quantity makes the first program smaller. A future variable-supply extension needs a cap, validated inputs, arithmetic checks and tests proving the new rules.

04 SOLANA DESIGN

PDAs and deterministic addresses

An address derived by a program is not automatically a permission to mint.



A Program Derived Address is found deterministically from seeds, a program ID and a bump. It has no ordinary private key. The owning program can authorise CPIs for one of its PDAs by supplying the matching seeds. Creating or deriving an address alone does not allocate an account or give it token permissions. [R38]

Derive exactly the address the Rust constraints expect

```
const [mint] = PublicKey.findProgramAddressSync(
    [Buffer.from("mint"), creator.toBuffer()], program.programId,
);
const treasury = getAssociatedTokenAddressSync(mint, creator);
```

In this lab, the mint address is a PDA, but the temporary mint authority is the creator wallet. The creator already signs initialize, and that signature authorises mint_to and set_authority CPIs. The program does not use the mint PDA as a long-lived mint authority. This keeps address derivation and permission control visible as separate concepts.

Mismatch	Likely cause
Client and program derive different mint addresses.	Different seed label, creator, program ID or key encoding.
ATA does not match.	Different owner, mint or Token Program ID.
PDA changes after a rebuild.	Program keypair / program ID was regenerated rather than preserved.

Checkpoint

Record the derived mint before initialization. After the transaction, confirm the created account is at that address. Do not repair a mismatch by removing the seeds constraint.

04 SOLANA DESIGN

Supply, fees and limits

Use precise units and separate network fees from account-storage funding.

Figure	Value	Source / meaning
Lamports per SOL	1,000,000,000.	Unit conversion used for SOL balances and fees.
Base fee	5,000 lamports per signature = 0.000005 SOL.	Current Solana fee documentation. [R41]
Maximum transaction CU limit	1,400,000 compute units.	Current documented execution limit. [R41]
Legacy / v0 transaction size	1,232 bytes.	The client formats used by this lab; not an arbitrary instruction count. [R61]
New v1 transaction size	4,096 bytes.	Current docs describe a distinct v1 format; the lab does not adopt it. [R61]
JavaScript safe integer	$2^{53} - 1 = 9,007,199,254,740,991$.	Use bigint for raw token quantities.
u64 maximum	$2^{64} - 1 = 18,446,744,073,709,551,615$.	Integer storage boundary, not a suitable business supply policy. [R12]
Course supply	1,000,000 tokens at 6 decimals.	A teaching assumption: 1,000,000,000,000 raw units.

A small fee example

For the legacy/v0 transaction format used by the web3.js client, an optional compute price of 1,000 micro-lamports per CU and a requested limit of 200,000 CU gives a priority fee of 200 lamports. With one signature and the documented base fee, that is 5,200 lamports, or 0.0000052 SOL. This is arithmetic illustration, not an estimate of total program deployment cost. [\[R62\]](#)

Account funding is a separate budget

New accounts need the minimum balance required for their storage, and program deployment funds much larger program-related accounts. Query the current network requirements rather than quoting a fixed fiat price. A successful token creation may debit more SOL than its transaction fee because it also creates the mint and treasury account.

The devnet lab initially requests two test SOL. That is a starting request, not a guarantee that all deployments will fit. Check your balance and the deployment error's required funding. Use the official devnet faucet if public airdrops throttle. A ledger reset removes locally funded accounts.

Budget record

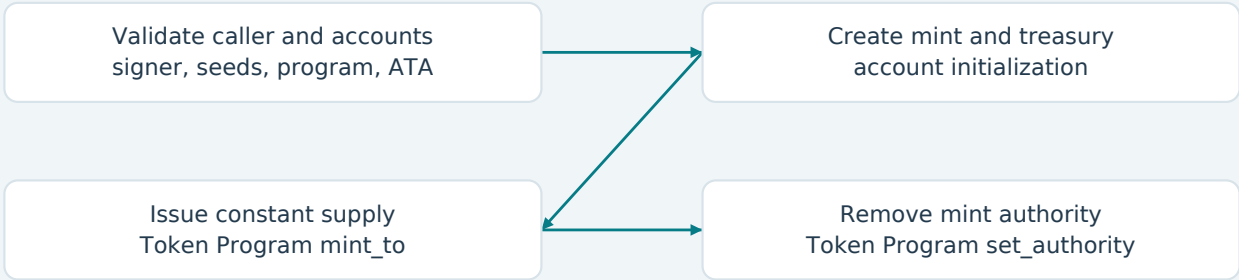
Record balance before and after each lab step, transaction signature and accounts created. Keep fiat valuations out of the technical acceptance criteria: token price does not prove program correctness.

04 SOLANA DESIGN

Transactions and atomicity

Why supply creation and authority removal belong in one operation.

FIGURE 16 / The initialize transaction



Every state change commits together; a failed instruction reverts state changes. [R40]

A transaction groups instructions that the network processes together. The custom initialize instruction also invokes standard programs through CPI. If a required operation fails, the transaction's state changes are reverted; fees can still be charged. Atomicity closes the window that would exist between a separate minting transaction and a later authority-removal transaction. [R40] [R39]

Situation	Required outcome
All validations and CPIs succeed.	Mint and treasury exist; supply is exact; mint/freeze authorities are absent.
Account validation fails.	No successful initialization is recorded; intended mint remains uncreated.
Mint or revoke CPI fails.	The transaction does not retain partial issuance state.
Caller loses the RPC response.	Inspect signature and chain state before deciding whether to retry.
Caller repeats initialize.	Already-created mint prevents another initialization for that creator/program.

A transaction signature is an identifier, not a success certificate. Read confirmation and transaction errors, then check the postconditions. The initialize script first checks whether the expected mint already exists so that rerunning an operator step produces an inspection rather than intentionally issuing again.

A useful mental model

The custom program defines an allowed sequence. The Token Program enforces mint authority and balance rules. Both sets of checks are needed; the frontend cannot substitute for either.

05 BUILD THE TOKEN PROJECT

Lab 1: create a standard token

Learn the existing Token Program before writing custom issuance logic.

Working directory: a new scratch folder for CLI practice. In terminal A, start a local validator. Keep it running while you use terminal B. A fresh folder avoids reusing another course's ledger. Local account addresses and transaction signatures will differ from this manual.

Terminal A - start the isolated local ledger

```
mkdir -p solana-cli-lab
cd solana-cli-lab
solana-test-validator
```

Terminal B - configure and fund the lab wallet

```
solana config set --url localhost
solana config set --keypair "$HOME/.config/solana/ai-coding-lab.json"
solana config get
solana airdrop 10
solana balance
spl-token create-token --decimals 6
```

The final command prints the new mint address. Copy your address into the variable below. The CLI's mint amount is a human-facing token quantity; supplying 1,000,000 here creates 1,000,000 tokens at six decimals, not 1,000,000 raw units. The standard lifecycle is documented in Solana's quickstart. [\[R30\]](#)

Terminal B - create the treasury and initial supply

```
LAB_MINT="REPLACE_WITH_THE_PRINTED_MINT"
spl-token create-account "$LAB_MINT"
spl-token mint "$LAB_MINT" 1000000
spl-token supply "$LAB_MINT"
spl-token display "$LAB_MINT"
```

Observe	Record in your evidence
Mint creation	Mint address, decimals and transaction signature.
Token-account creation	The wallet's ATA address and its owning wallet.
Minting	Displayed supply of 1,000,000 and the minting transaction.
Authority	Mint authority still exists until the next page's revocation step.

Completion gate

Explain which program owns the mint, where the balance is stored and why a wallet needs a token account. Do not treat the newly created token as a custom smart contract.

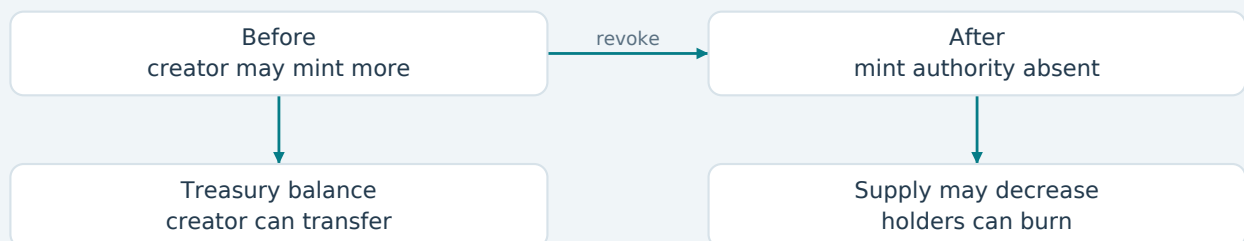
05 BUILD THE TOKEN PROJECT

Lab 1: lock supply and transfer

Observe authority changes and supply conservation directly.

Terminal B - remove mint authority after checking the token

```
spl-token display "$LAB_MINT"  
spl-token authorize "$LAB_MINT" mint --disable  
spl-token display "$LAB_MINT"  
# Intentionally fails: mint authority has been removed.  
spl-token mint "$LAB_MINT" 1
```

FIGURE 17 / Permissions before and after revocation

Revoking minting does not revoke transfer rights or prevent holder burns. [\[R36\]](#) [\[R35\]](#)

This is a local training mint, so permanent authority removal is part of the exercise. Check that freeze authority is also absent: the core create-token command did not enable it. If you created a different mint with a freeze authority, investigate and remove that authority deliberately while it is still available. Do not repeatedly run a removal command against a field already set to none.

Optional local transfer - replace the recipient with a second test wallet

```
LAB_RECIPIENT="REPLACE_WITH_A_SECOND_TEST_PUBLIC_KEY"  
spl-token transfer "$LAB_MINT" 25 "$LAB_RECIPIENT" \  
  --fund-recipient --allow-unfunded-recipient  
spl-token supply "$LAB_MINT"
```

The recipient must have a valid test-wallet public key. The transfer flags permit and fund the recipient's token account where needed; consult `spl-token transfer --help` for your installed CLI. Confirm that total supply is unchanged and balances moved by 25 tokens. [\[R34\]](#)

Before moving to Lab 2

Stop terminal A with Ctrl-C. Anchor will start a separate validator for integration tests. Do not leave another process using RPC port 8899.

05 BUILD THE TOKEN PROJECT

Lab 2: scaffold the project

Use the supplied files, then generate your own program identity.

Main route: extract the companion ZIP and open its `fixed_supply` folder. It contains the Rust program, manifests, TypeScript scripts, tests and read-only React app. No private key or real program ID is supplied. The System Program address in the template is a conspicuous placeholder that must be replaced before any meaningful build or deployment.

From the `fixed_supply` project root - initial setup

```
mkdir -p target/deploy
solana-keygen new --outfile target/deploy/fixed_supply-keypair.json
anchor keys sync
npm install
# Inspect lib.rs and both cluster entries in Anchor.toml.
anchor keys list
```

Run the key generation only when the file does not exist. A regenerated program keypair changes your program ID and every mint PDA derived under that ID. Preserve the program identity securely. After keys sync, verify both localnet and devnet entries; update an entry manually if the installed CLI did not synchronise it.

File / directory	Role
<code>programs/fixed_supply/src/lib.rs</code>	The complete on-chain implementation.
<code>programs/fixed_supply/Cargo.toml</code>	Program crate, Anchor dependencies and IDL build features.
<code>Anchor.toml</code>	Cluster addresses, provider and test command.
<code>scripts/</code>	Shared client setup, initialize, inspect and transfer.
<code>tests/fixed_supply.ts</code>	Local integration checks, including account substitution and re-mint rejection.
<code>app/</code>	Separate npm project for the read-only dashboard.
<code>target/idl</code> and <code>target/types</code>	Generated public program interface; appears after build.

Optional: explore an empty Anchor scaffold

To understand the generator separately, use `anchor init blank_training --template single --test-template mocha` in another directory. Current Anchor also offers a modular template and Rust-based test templates. The single-file program is selected here to make the first instruction easy to read. [\[R20\]](#)

Completion gate

The generated program key, `declare_id` value and both `Anchor.toml` cluster values agree. The lab wallet remains a separate file and public address.

05 BUILD THE TOKEN PROJECT

Configuration and build outputs

Your provider, dependencies and generated interface must agree.

Anchor.toml - key settings; replace YOUR_PROGRAM_ID

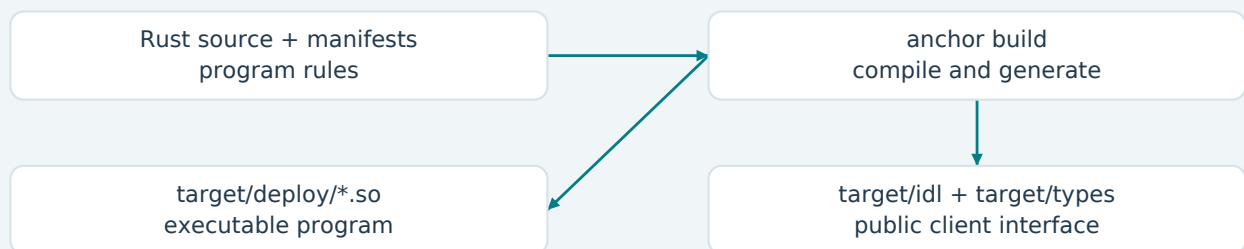
```
[toolchain]
anchor_version = "1.2.0"

[programs.localnet]
fixed_supply = "YOUR_PROGRAM_ID"

[programs.devnet]
fixed_supply = "YOUR_PROGRAM_ID"

[provider]
cluster = "Localnet"
wallet = "~/.config/solana/ai-coding-lab.json"

[scripts]
test = "npm test"
```

FIGURE 18 / Build artifacts and identity

The generated IDL address must match the program you actually deploy. [\[R23\]](#)

The supplied crate pins anchor-lang and anchor-spl to =1.2.0, declares token and associated_token features, and includes idl-build features. The root npm project uses @anchor-lang/core 1.2.0, web3.js v1, SPL Token 0.4, TypeScript 5.9 and Node-based Mocha tests. The exact dependency trees become lockfiles after your first resolution.

Project root - compile and check the client

```
anchor build
npm run typecheck
solana-keygen pubkey target/deploy/fixed_supply-keypair.json
```

The host Rust toolchain and the SBF build toolchain have different roles. If a build error mentions platform tools or a lockfile format, compare the versions with Anchor's CLI reference before changing source code. Ensure all public program IDs agree before continuing. [\[R20\]](#) [\[R21\]](#)

05 BUILD THE TOKEN PROJECT

Program code: issuance

Part 1 of programs/fixed_supply/src/lib.rs. Join with the next page.

programs/fixed_supply/src/lib.rs - part 1, lines 1-40

```

use anchor_lang::prelude::*;
use anchor_spl::{
    associated_token::AssociatedToken,
    token::{self, Mint, MintTo, SetAuthority, Token, TokenAccount},
};
use anchor_spl::token::spl_token::instruction::AuthorityType;

// Training placeholder. Run anchor keys sync before building.
declare_id!("11111111111111111111111111111111");

pub const DECIMALS: u8 = 6;
pub const INITIAL_SUPPLY: u64 = 1_000_000_000_000;

#[program]
pub mod fixed_supply {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
        let mint_accounts = MintTo {
            mint: ctx.accounts.mint.to_account_info(),
            to: ctx.accounts.treasury.to_account_info(),
            authority: ctx.accounts.creator.to_account_info(),
        };
        token::mint_to(
            CpiContext::new(ctx.accounts.token_program.key(), mint_accounts),
            INITIAL_SUPPLY,
        )?;

        let revoke_accounts = SetAuthority {
            current_authority: ctx.accounts.creator.to_account_info(),
            account_or_mint: ctx.accounts.mint.to_account_info(),
        };
        token::set_authority(
            CpiContext::new(ctx.accounts.token_program.key(), revoke_accounts),
            AuthorityType::MintTokens,
            None,
        )?;
        Ok(())
    }
}

```

The constants encode the course's supply policy. `initialize` constructs the accounts for `mint_to`, issues exactly the raw-unit constant, then removes `MintTokens` authority. A question mark propagates a CPI error so the operation cannot report success after a failed issuance or revocation. [\[R24\]](#)

`CpiContext::new` receives the validated Token Program public key in Anchor 1.x. Its account structures provide the mint, destination and current authority. The creator signature already authorises those operations; no separate PDA mint-authority signing seeds are required by this design. [\[R26\]](#) [\[R27\]](#)

05 BUILD THE TOKEN PROJECT

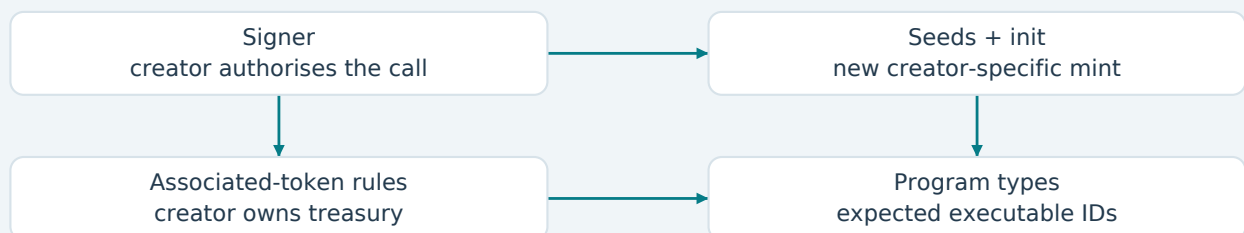
Program code: account validation

Part 2 of programs/fixed_supply/src/lib.rs. These constraints enforce the design.

programs/fixed_supply/src/lib.rs - part 2, lines 41-67

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub creator: Signer<'info>,
    #[account(
        init,
        payer = creator,
        seeds = [b"mint", creator.key().as_ref()],
        bump,
        mint::decimals = DECIMALS,
        mint::authority = creator,
        mint::token_program = token_program,
    )]
    pub mint: Account<'info, Mint>,
    #[account(
        init,
        payer = creator,
        associated_token::mint = mint,
        associated_token::authority = creator,
        associated_token::token_program = token_program,
    )]
    pub treasury: Account<'info, TokenAccount>,
    pub token_program: Program<'info, Token>,
    pub associated_token_program: Program<'info, AssociatedToken>,
    pub system_program: Program<'info, System>,
}
```

FIGURE 19 / What the account constraints prove



Validation happens before the handler's successful issuance sequence. [\[R22\]](#) [\[R25\]](#)

init creates the mint and ATA, funded by the creator. The mint constraint specifies six decimals and a temporary creator mint authority. Omitting mint::freeze_authority leaves it absent. Program<Token> binds the call to the original Token Program. The deterministic mint and init prevent repeated initialization for the same creator and program.

05 BUILD THE TOKEN PROJECT

Tests: setup and wrong accounts

Part 1 of tests/fixed_supply.ts. The suite uses a fresh local ledger.

tests/fixed_supply.ts - part 1, lines 1-43

```
import * as anchor from "@anchor-lang/core";
import { expect } from "chai";
import { Keypair, PublicKey, SystemProgram } from "@solana/web3.js";
import {
  getMint, getAccount, getAssociatedTokenAddressSync, mintTo,
  TOKEN_PROGRAM_ID, ASSOCIATED_TOKEN_PROGRAM_ID,
} from "@solana/spl-token";
import type { FixedSupply } from "../target/types/fixed_supply";

describe("fixed_supply", () => {
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);
  const program = anchor.workspace.FixedSupply as anchor.Program<FixedSupply>;
  const creator = provider.wallet.publicKey;
  const [mint] = PublicKey.findProgramAddressSync(
    [Buffer.from("mint"), creator.toBuffer()], program.programId,
  );
  const treasury = getAssociatedTokenAddressSync(mint, creator);
  const accounts = {
    creator, mint, treasury, tokenProgram: TOKEN_PROGRAM_ID,
    associatedTokenProgram: ASSOCIATED_TOKEN_PROGRAM_ID,
    systemProgram: SystemProgram.programId,
  };

  async function mustReject(action: () => Promise<unknown>, pattern: RegExp) {
    let caught: unknown;
    try { await action(); } catch (error) { caught = error; }
    expect(caught, "Expected rejection").not.to.equal(undefined);
    const error = caught as { message?: string; logs?: string[] };
    const evidence = String(error.message) + "\n" + (error.logs ?? []).join("\n");
    expect(evidence).to.match(pattern);
  }

  it("rejects a substituted mint PDA before initialization", async () => {
    await mustReject(
      () => program.methods.initialize().accountsStrict({
        ...accounts, mint: Keypair.generate().publicKey,
      }).rpc(),
      /ConstraintSeeds|seeds constraint|0x7d6/i,
    );
    expect(await provider.connection.getAccountInfo(mint)).to.equal(null);
  });
});
```

The helper `mustReject` requires both rejection and a relevant diagnostic category. A connection failure must not count as evidence that an account constraint worked. The first test substitutes a different mint and checks that the intended mint remains uncreated.

Fresh local state is a test assumption: the positive initialization later in this file expects the creator mint not to exist yet. Preserve that ordering or provide an isolated creator and ledger per test. Do not run this initialization suite unchanged against a persistent devnet mint.

05 BUILD THE TOKEN PROJECT

Tests: state and postconditions

Part 2 of tests/fixed_supply.ts. Join all three listings in order.

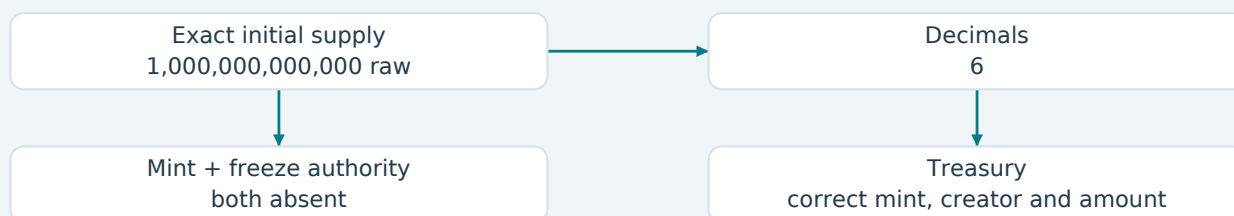
tests/fixed_supply.ts - part 2, lines 44-78

```
it("rejects a substituted token program before initialization", async () => {
  await mustReject(
    () => program.methods.initialize().accountsStrict({
      ...accounts, tokenProgram: SystemProgram.programId,
    }).rpc(),
    /InvalidProgramId|program id|incorrect program|0xbc0/i,
  );
  expect(await provider.connection.getAccountInfo(mint)).to.equal(null);
});

it("rejects a substituted treasury ATA before initialization", async () => {
  await mustReject(
    () => program.methods.initialize().accountsStrict({
      ...accounts, treasury: Keypair.generate().publicKey,
    }).rpc(),
    /ConstraintAssociated|associated|0x7d9/i,
  );
  expect(await provider.connection.getAccountInfo(mint)).to.equal(null);
});

it("creates the exact supply and revokes minting", async () => {
  await program.methods.initialize().accountsStrict(accounts).rpc();
  const state = await getMint(provider.connection, mint);
  const balance = await getAccount(provider.connection, treasury);
  expect(state.decimals).to.equal(6);
  expect(state.supply).to.equal(1_000_000_000_000n);
  expect(state.mintAuthority).to.equal(null);
  expect(state.freezeAuthority).to.equal(null);
  expect(balance.amount).to.equal(1_000_000_000_000n);
  expect(balance.mint.equals(mint)).to.equal(true);
  expect(balance.owner.equals(creator)).to.equal(true);
  const info = await provider.connection.getAccountInfo(mint);
  expect(info?.owner.equals(TOKEN_PROGRAM_ID)).to.equal(true);
});
```

FIGURE 20 / Required successful-state checks



Inspect decoded chain state; a frontend label cannot prove these properties.

The wrong-program and wrong-treasury tests run before the positive case. The positive case checks every important postcondition, including the mint's account-level Token Program owner.

05 BUILD THE TOKEN PROJECT

Tests: repeats and further minting

Part 3 of tests/fixed_supply.ts. Failures must preserve the supply.

tests/fixed_supply.ts - part 3, lines 79-97

```
it("rejects initialization of the same mint twice", async () => {
  await mustReject(
    () => program.methods.initialize().accountsStrict(accounts).rpc(),
    /already in use|already initialized|custom program error: 0x0/i,
  );
  expect((await getMint(provider.connection, mint)).supply)
    .to.equal(1_000_000_000_000n);
});

it("the Token Program refuses further minting", async () => {
  const payer = (provider.wallet as anchor.Wallet).payer;
  await mustReject(
    () => mintTo(provider.connection, payer, mint, treasury, payer, 1n),
    /fixed supply|FixedSupply|0x5/i,
  );
  expect((await getMint(provider.connection, mint)).supply)
    .to.equal(1_000_000_000_000n);
});
```

Why these two tests are different

Repeated initialization tests the custom program's deterministic account-creation rule. The further minting test calls the Token Program through the SPL client, independently of the custom program. This proves that the mint's own permission state rejects additional issuance, rather than merely showing that the custom program lacks a second mint instruction.

Check	What it demonstrates	What it does not demonstrate
Wrong mint / treasury / program	Account substitution is rejected for the tested cases.	A comprehensive audit of all possible program interactions.
Exact state after initialize	This execution created the specified supply and authority state.	Metadata or exchange support.
Repeated initialization	The same deterministic mint is not re-created.	Only one token exists for every possible creator.
Direct re-mint attempt	No additional original-program issuance is permitted for this mint.	Holders cannot burn or transfer units.

Extend the test suite as your design grows

If you add a mutable state account, test its initial values, space and access rules. If you add a variable supply, test the cap, zero, overflow and precision boundaries. If you add metadata, test its authority and failure behavior before revocation. New behavior needs new checks, but avoid tests that merely repeat a constant from the implementation without challenging any outcome.

05 BUILD THE TOKEN PROJECT

Run the local gates

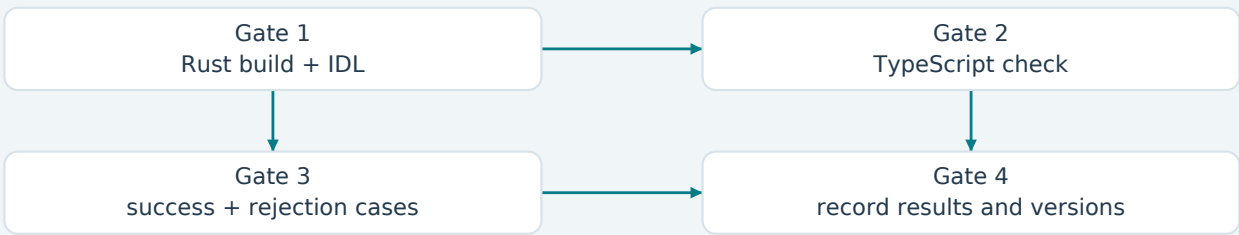
Build, check types and execute the integration suite before deployment.

Project root - local acceptance commands

```
solana config set --url localhost
anchor --version
solana --version
anchor build
npm run typecheck
anchor test --validator legacy
```

The explicit legacy flag selects solana-test-validator instead of Anchor 1.x's default Surfpool backend. Use a compatible Agave 4.x validator for the default v3 SBF build. Make sure no other local validator is listening on port 8899. Anchor manages the local validator, deploys the program and executes the test command configured in Anchor.toml. [R57]

FIGURE 21 / The local acceptance gates



A later gate does not excuse a failed earlier check.

Result	Action
Missing tool / incompatible build architecture	Repair the toolchain and preserve source requirements.
Type check fails	Compare generated types, imports and installed Anchor client.
A negative test reports connection failure	Repair the network setup; it has not tested the constraint yet.
Already-created mint during the positive case	Use a fresh isolated ledger or correctly reset the test environment.
All required checks pass	Create a Git checkpoint and proceed to the separate devnet workflow.

Record the actual commands and outputs in docs/lab-evidence.md. The authored scaffold is not a substitute for your local results: it was not compiled or deployed in this document's authoring environment.

05 BUILD THE TOKEN PROJECT

Deploy the program to devnet

A program deployment and a token initialization are separate transactions.

Devnet is a shared test environment. Before deployment, confirm the local checks passed and preserve the program keypair used by your build. The deployed program ID must be the same ID used in `declare_id`, `Anchor.toml` and the generated IDL. Deployment alone does not create the token mint.

Project root - explicit devnet configuration and deployment

```
solana config set --url devnet
solana config set --keypair "$HOME/.config/solana/ai-coding-lab.json"
solana config get
solana address
solana airdrop 2
solana balance
anchor deploy --provider.cluster devnet
solana program show YOUR_PROGRAM_ID --url devnet
```

Replace `YOUR_PROGRAM_ID` with the program ID recorded from anchor keys list. If the deployment needs more account funding than your balance, request additional test SOL through the official faucet or resolve rate limiting. Deployment funding depends on the artifact and network state; two SOL is a starting allowance, not a guaranteed budget. [\[R42\]](#)

Record	Why it matters
Source commit	Identifies the implementation intended for this deployment.
Program ID and cluster	Identifies the executable clients will call.
Deployment result / signature	Provides a network record for investigation.
Upgrade authority	Shows who can replace the program's code.
Build and dependency versions	Makes later compatibility failures easier to reproduce.

Keep cluster configuration explicit

The `--provider.cluster` override selects devnet for Anchor deployment. It does not automatically export environment variables for later Node scripts. On the next page set both `ANCHOR_PROVIDER_URL` and `ANCHOR_WALLET`. If you deploy using a different approved wallet, update the provider intentionally and record the change.

Completion gate

The network program inspection shows your expected program ID and upgrade authority. The mint has not been initialized until the separate initialize script succeeds.

05 BUILD THE TOKEN PROJECT

Initialize the token on devnet

The operator script is careful about repeats and checks existing state.

Project root - select the script provider

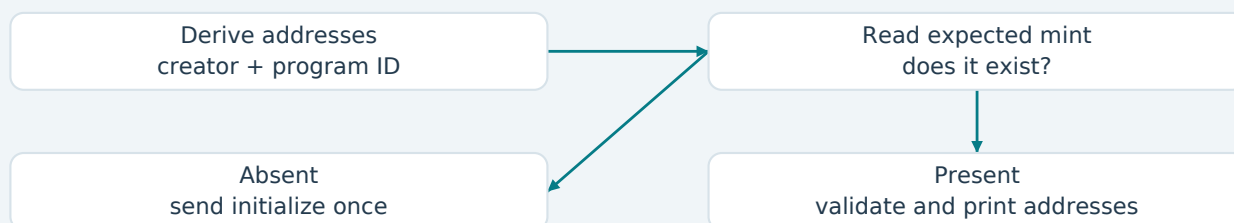
```
export ANCHOR_PROVIDER_URL=https://api.devnet.solana.com
export ANCHOR_WALLET="$HOME/.config/solana/ai-coding-lab.json"
npm run initialize
```

scripts/initialize.ts - complete file

```
import { addresses, program, provider, requireLabCluster } from "../common";
import { getMint } from "@solana/spl-token";

async function main() {
  requireLabCluster();
  const accounts = addresses(provider.wallet.publicKey);
  const existing = await provider.connection.getAccountInfo(accounts.mint);
  if (existing) {
    const mint = await getMint(provider.connection, accounts.mint);
    if (mint.decimals !== 6 || mint.mintAuthority !== null ||
        mint.freezeAuthority !== null || mint.supply > 1_000_000_000_000n) {
      throw new Error("Existing mint does not satisfy the lab invariants");
    }
    console.log("Already initialized; no new transaction sent");
  } else {
    const signature = await program.methods.initialize()
      .accountsStrict(accounts).rpc();
    console.log("Transaction:", signature);
  }
  console.log("Program:", program.programId.toBase58());
  console.log("Creator:", accounts.creator.toBase58());
  console.log("Mint:", accounts.mint.toBase58());
  console.log("Treasury:", accounts.treasury.toBase58());
}
main().catch((error) => { console.error(error); process.exitCode = 1; });
```

FIGURE 22 / Initialize or inspect existing state



The script's existence check improves operator behavior; program constraints remain the final guard.

The shared setup in `scripts/common.ts` loads the generated IDL, constructs the provider, derives accounts and limits writes to localhost or public devnet. The complete shared file is supplied in the ZIP. Record the initialization signature, mint and treasury printed by this command.

05 BUILD THE TOKEN PROJECT

Inspect the token independently

Read account ownership, supply and permissions after initialization.

Project root - uses the script provider already selected

```
npm run inspect
```

scripts/inspect.ts - complete file

```
import { addresses, provider } from "../common";
import { getAccount, getMint, TOKEN_PROGRAM_ID } from "@solana/spl-token";

async function main() {
  const { mint, treasury, creator } = addresses(provider.wallet.publicKey);
  const info = await provider.connection.getAccountInfo(mint, "confirmed");
  if (!info || !info.owner.equals(TOKEN_PROGRAM_ID)) {
    throw new Error("Missing mint or unexpected owning program");
  }
  const state = await getMint(provider.connection, mint, "confirmed");
  const balance = await getAccount(provider.connection, treasury, "confirmed");
  if (!balance.mint.equals(mint) || !balance.owner.equals(creator)) {
    throw new Error("Treasury mint/owner mismatch");
  }
  if (state.decimals !== 6 || state.mintAuthority !== null ||
    state.freezeAuthority !== null || state.supply > 1_000_000_000_000n) {
    throw new Error("Mint invariant failed");
  }
  console.log(JSON.stringify({
    cluster: provider.connection.rpcEndpoint,
    mint: mint.toBase58(),
    rawSupply: state.supply.toString(),
    decimals: state.decimals,
    mintAuthority: state.mintAuthority,
    freezeAuthority: state.freezeAuthority,
    rawTreasuryBalance: balance.amount.toString(),
  }, null, 2));
}
main().catch((error) => { console.error(error); process.exitCode = 1; });
```

Immediately after initialization, raw supply and raw treasury balance should each equal 1,000,000,000,000. The supply invariant later becomes 'no more than the initial supply' because holder burns may reduce it. Treasury balance can also fall after a transfer.

Open [Solana Explorer](#), choose devnet and search the public mint address. A mint's explorer page and this decoded script output should agree. The script additionally validates the Token Program owner and treasury relationship.

Do not confuse identifiers

Program ID identifies the custom executable. Mint identifies this token. Treasury identifies the creator's token account. A successful deployment result does not prove correct mint state.

05 BUILD THE TOKEN PROJECT

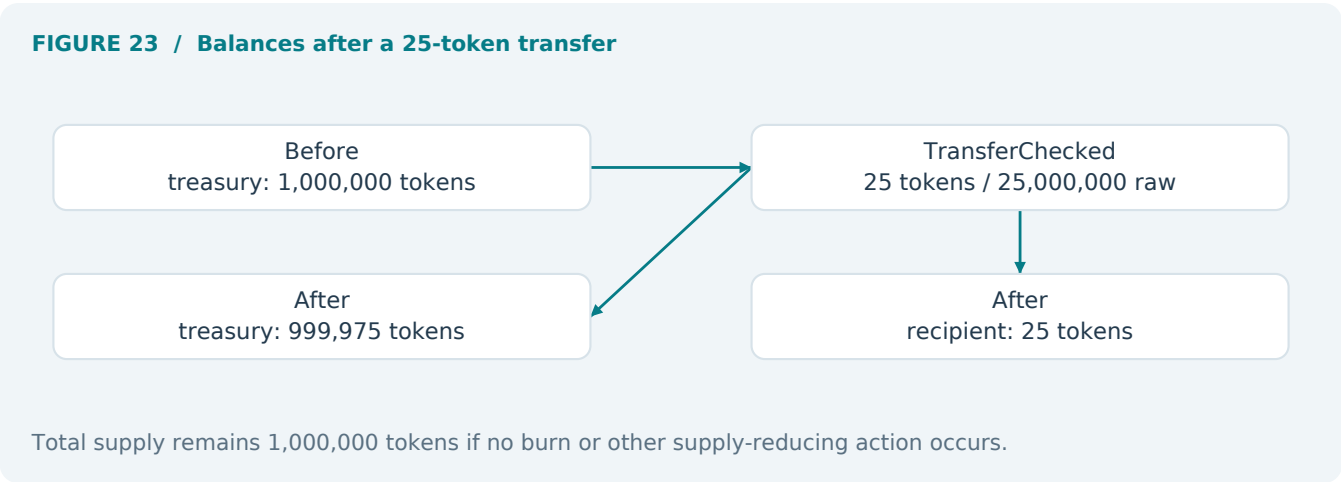
Transfer and reconcile balances

Move tokens with checked decimals and confirm supply conservation.

The complete scripts/transfer.ts file is in the ZIP. It validates a recipient and raw integer amount, loads only the local signing key, checks the lab cluster and mint invariants, creates the recipient ATA if needed, then uses transferChecked. The account creation may be a separate transaction from the transfer; both results need appropriate failure handling. [R45] [R46]

Project root - transfer 25 display tokens to a second test wallet

```
npm run transfer -- YOUR_RECIPIENT_PUBLIC_KEY 25000000
npm run inspect
```



Reconciliation check	Expected result
Creator treasury delta	Minus 25,000,000 raw units.
Recipient token-account delta	Plus 25,000,000 raw units.
Mint supply delta	Zero.
Mint / freeze authority	Still absent.
Recipient identity	The intended public key, not merely a valid address string.

Keep the recipient test keypair separate and record its public key. Inspect the recipient ATA through the explorer or an additional getAccount read. Rerunning the transfer sends another transfer: it is not idempotent. If a response is lost, investigate the signature and balances before sending again.

Completion gate

Your evidence shows both balance deltas, unchanged supply and the transfer signature. Explain why the CLI's 25-token quantity and this script's 25000000 argument represent the same value.

06 APPLICATION AND LAUNCH

Build the read-only dashboard

The frontend teaches data retrieval without introducing wallet signing.

The dashboard is a separate application in `app/`. Its purpose is to read public devnet supply and provide a correctly configured explorer link. It cannot initialize, transfer, burn or sign transactions. This separation lets you learn frontend state and error handling without exposing the operator wallet to the browser.

Project root - install and configure the supplied app

```
npm --prefix app install
cp app/.env.example app/.env.local
# Edit app/.env.local in the editor:
# VITE_MINT_ADDRESS must be your initialized devnet mint.
npm --prefix app run dev
```

Use the URL printed by Vite, normally `http://localhost:5173`. Restart the dev server after changing environment configuration. The app hardcodes public devnet RPC rather than accepting an arbitrary write endpoint. Its only configured value is a public mint address. [\[R07\]](#) [\[R08\]](#)

Frontend file	Purpose
<code>index.html</code>	HTML entry point and root element.
<code>src/main.tsx</code>	Mounts the React application into the root.
<code>src/App.tsx</code>	Displays loading, success and error states.
<code>src/rpc.ts</code>	Builds the RPC request and validates the returned supply.
<code>src/style.css</code>	Readable layout and responsive text wrapping.
<code>.env.example</code>	Documents the required public mint configuration.

Understand the UI's evidence limits

`getTokenSupply` reports amount and decimals. It does not prove that mint authority is absent or that the mint came from your intended custom program. The page says this explicitly and points the operator to the inspect script. Do not display a 'verified immutable token' badge based only on a successful supply read.

If you want to learn the scaffold generator

In a separate directory, run `npm create vite@latest dashboard_practice -- --template react-ts`. Compare the generated files with the supplied app rather than replacing your working lab. Vite documents a minimum Node requirement of 20.19+ or 22.12+; the supported Node 24 LTS selected here satisfies that baseline. [\[R07\]](#)

Completion gate - The browser shows your mint, current supply and devnet explorer link. The private key remains outside frontend code and configuration.

06 APPLICATION AND LAUNCH

Dashboard: validated RPC

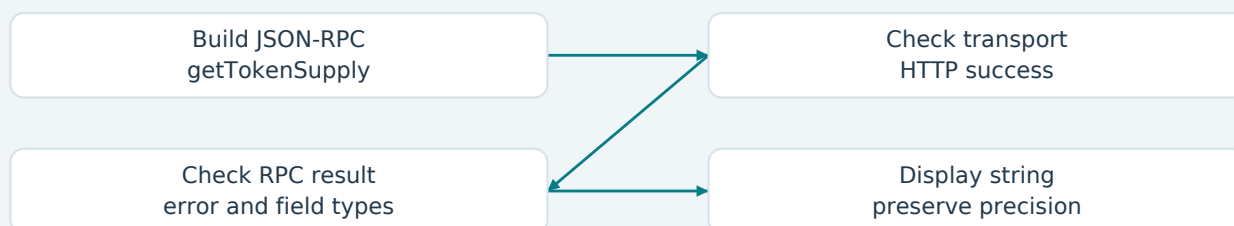
app/src/rpc.ts - complete file. Both transport and response shape are checked.

app/src/rpc.ts

```
type Supply = { amount: string; decimals: number; uiAmountString: string };
export const mint = import.meta.env.VITE_MINT_ADDRESS as string | undefined;
const endpoint = "https://api.devnet.solana.com";

function record(value: unknown): value is Record<string, unknown> {
  return typeof value === "object" && value !== null;
}

export async function readSupply(signal: AbortSignal): Promise<Supply> {
  if (!mint || !/^[1-9A-HJ-NP-Za-km-z]{32,44}$/.test(mint)) {
    throw new Error("Configure a base58 devnet mint address");
  }
  const response = await fetch(endpoint, {
    method: "POST", signal,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      jsonrpc: "2.0", id: 1, method: "getTokenSupply",
      params: [mint, { commitment: "confirmed" }],
    }),
  });
  if (!response.ok) throw new Error("RPC HTTP " + response.status);
  const data: unknown = await response.json();
  if (!record(data)) throw new Error("Malformed RPC response");
  if (data.error) throw new Error("RPC rejected the mint query");
  const result = data.result;
  if (!record(result) || !record(result.value)) {
    throw new Error("Missing supply data");
  }
  const value = result.value;
  if (typeof value.amount !== "string" || !/^[0-9]+$/.test(value.amount) ||
    typeof value.decimals !== "number" || value.decimals !== 6 ||
    typeof value.uiAmountString !== "string" ||
    !/^[0-9]+(\.[0-9]+)?$/.test(value.uiAmountString) ||
    BigInt(value.amount) > 1_000_000_000_000n) {
    throw new Error("Supply response does not match the training token");
  }
  return { amount: value.amount, decimals: value.decimals,
    uiAmountString: value.uiAmountString };
}
```

FIGURE 24 / The browser read path

Validate the getTokenSupply response; preserve raw-integer precision. [R65]

06 APPLICATION AND LAUNCH

Dashboard: state and presentation

app/src/App.tsx - complete file. The supporting CSS and entry point are in the ZIP.

app/src/App.tsx

```
import { useEffect, useState } from "react";
import { mint, readSupply } from "./rpc";
import "./style.css";

export default function App() {
  const [supply, setSupply] = useState<string | null>(null);
  const [error, setError] = useState("");
  useEffect(() => {
    const controller = new AbortController();
    readSupply(controller.signal)
      .then((value) => setSupply(value.uiAmountString))
      .catch((failure: unknown) => {
        if (!controller.signal.aborted) {
          setError(failure instanceof Error ? failure.message : "Request failed");
        }
      });
    return () => controller.abort();
  }, []);
  return <main>
    <p className="tag">ENGINEERING TRAINING / DEVNET</p>
    <h1>Solana Training Token</h1>
    <p>This page reads public token supply. It cannot sign transactions.</p>
    <dl>
      <dt>Network</dt><dd>Solana devnet</dd>
      <dt>Mint</dt><dd>{mint ?? "Not configured"}</dd>
      <dt>Current supply</dt>
      <dd aria-live="polite">{supply ?? (error ? "Unavailable" : "Loading...")}</dd>
      <dt>Decimals</dt><dd>6</dd>
    </dl>
    {error && <p role="alert">{error}</p>}
    {mint && <a target="_blank" rel="noreferrer"
      href={"https://explorer.solana.com/address/" + encodeURIComponent(mint) +
        "?cluster=devnet"}>View mint on Solana Explorer</a>}
    <p>Supply shown here does not establish mint or freeze authority status.
      Run the inspect script to check those fields.</p>
  </main>;
}
```

useEffect starts the network request after the component mounts and aborts it on cleanup. State holds either the display quantity or an error. React renders text values as content rather than asking you to build arbitrary HTML strings. The UI reports unavailable data instead of pretending a failed request means zero supply. [R06]

The mint address wraps on narrow screens. The loading value uses an aria-live region, and errors use an alert role. These small choices make the dashboard clearer for users and assistive technology.

Keep the claim accurate

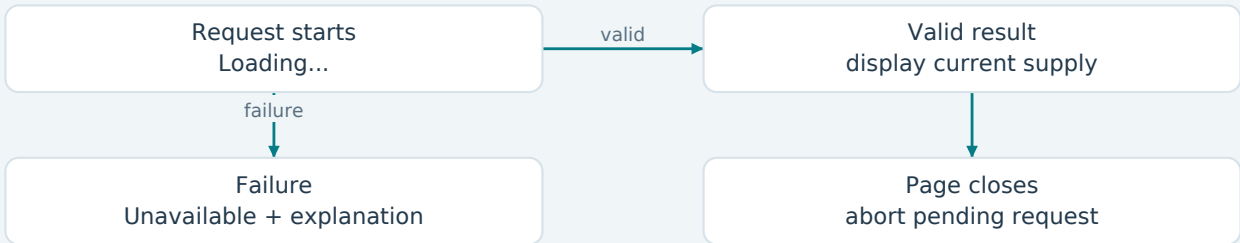
The 'Solana Training Token' title is an application label. The core lab has not created name, symbol or logo metadata on-chain.

06 APPLICATION AND LAUNCH

Application verification

A working development server is only one check.

FIGURE 25 / Observable dashboard states



Every state should be understandable without reading implementation details.

Project root - check the production build and preview

```
npm --prefix app run build
npm --prefix app run preview
```

Scenario	Expected behavior	Evidence
Configured existing devnet mint	Correct address, decimals and display supply.	Compare with inspect output and explorer.
Missing / malformed config	Visible configuration error.	Temporarily remove the env value and restart.
Nonexistent devnet mint	RPC error becomes an unavailable state.	Use a valid test public key with no mint account.
Offline or throttled RPC	HTTP / request failure becomes a visible error.	Disconnect locally or inspect a known throttled response.
Narrow viewport	Address wraps; key information remains legible.	Browser responsive view.
Production preview	Same behavior from the built output.	Run build, then preview; check the mint value.

Changing environment variables after a static build does not rewrite that build. Rebuild the app with the intended public configuration. A later production frontend deployment would publish the app assets, not the Rust executable. Treat frontend and on-chain release records as related but separate.

Further exercise

Add a refresh button with disabled/loading behavior and a clear last-updated timestamp. Avoid background polling until you have defined its interval, retry limits and public-RPC load.

06 APPLICATION AND LAUNCH

Metadata is a separate extension

Names, symbols and images do not replace the mint's identity.

The core project intentionally creates an original-program mint with no on-chain descriptive metadata. Many wallets and explorers rely on metadata to show a name, symbol and image. Metaplex Token Metadata is one established mechanism for associating those details with a mint. It introduces its own accounts, creation requirements and update authority. Metaplex currently labels it legacy but supported; check the chosen fungible-mint integration before extending this lab. [R44]

Metadata element	Meaning	Engineering consideration
Name and symbol	Human-facing labels.	Neither is unique; always retain the mint address.
URI	Location of off-chain descriptive JSON.	Availability and integrity of hosted content matter.
Image	Brand / display asset referenced by metadata.	Use an owned asset and durable hosting.
Update authority	Permission to change metadata under its rules.	Separate from minting, freezing and program upgrades.
Mutable / immutable policy	Whether future metadata updates are allowed.	Decide before claiming immutability to users.

The order matters

Creating metadata for this kind of mint generally requires the mint authority's authorization. The core lab removes that authority before initialize completes, so you should not assume you can attach standard Metaplex metadata afterward. Plan the extension before initialization: create metadata while the required authority is available, then revoke mint authority, preferably within the same successful transaction. [R63]

How to specify a metadata extension

Add a separate requirement covering the exact program/SDK, name, symbol, URI, update authority and mutability policy. Verify the SDK's compatibility with the selected Anchor client family before installing it. If its transaction builders use another client family, keep that integration explicit instead of mixing incompatible account or transaction types.

Your tests must verify that the metadata belongs to the intended mint, unauthorized updates fail and a failed metadata step does not leave an unexpected issuance state. Test explorer display separately: wallet indexing and cached display are outside your program's direct control.

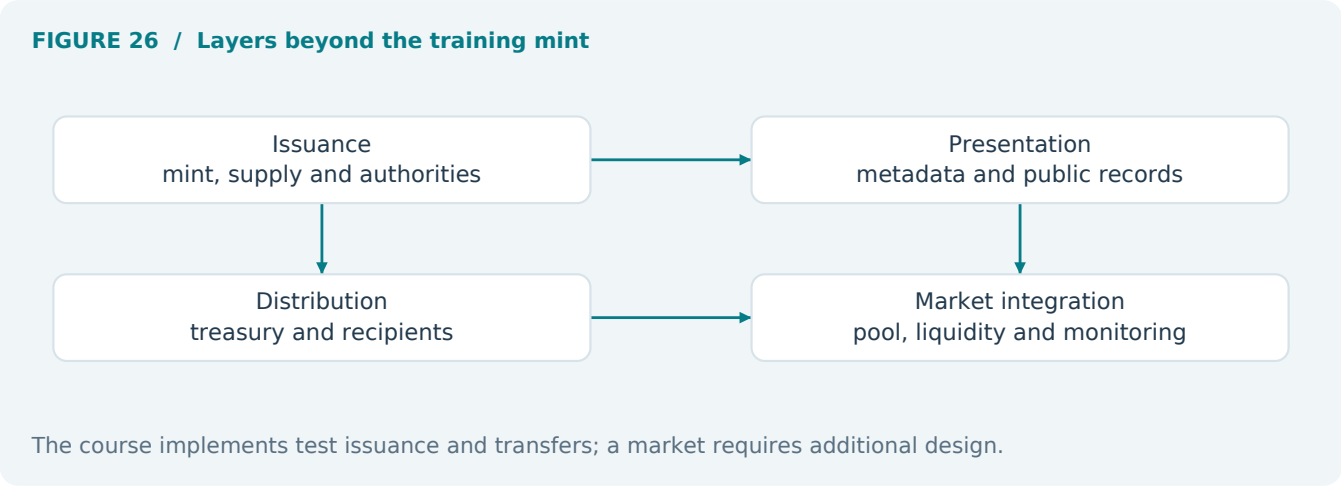
Important distinction

Setting a token title in React or uploading an image to a website does not create token metadata. Metadata also does not prove fair distribution, locked liquidity or token value.

06 APPLICATION AND LAUNCH

What a token launch includes

Issuance, distribution and a trading market are different deliverables.



Layer	Questions a project owner must answer
Supply and control	What is the mint? Who controls treasury, upgrades and any remaining authorities?
Distribution	Who receives units, what allocation is disclosed, and is vesting required?
Metadata	Which public details are stored, where are assets hosted and who can update them?
Liquidity	Which pool and quote asset are used, who funds it and who controls the liquidity position?
Operations	How are failures, compromised keys, RPC limits and user reports handled?
Public release	Which organisational, specialist and applicable legal reviews are needed for the proposed activity?

A pool can establish a ratio between two assets, but that ratio is not a guarantee of sustainable value or available exit liquidity. Do not describe mint creation as a trading launch. Publishing a dashboard also does not list the token on an exchange.

For your learning project, stop at devnet and write the missing layers as design exercises. A future real-value release needs a separate project brief, funding plan, custody design, independent review and concrete release evidence. This manual supplies technical concepts rather than a public-sale strategy.

Handover language
'Created and tested a fixed-supply token on devnet, with an inspection script and read-only dashboard' is an accurate description of the completed course outcome.

07 DELIVERY AND OPERATIONS

CI and repeatable checks

Automate the acceptance commands after you have reproduced them locally.

Continuous integration runs agreed checks in a predictable environment. The important part is the command and its prerequisites, not a green badge by itself. A fresh runner needs the compatible Solana toolchain, resolved lockfiles and disposable lab keys. The example below assumes an isolated preconfigured runner labelled `solana-training`; it is an optional workflow template, not a ready-made hosted runner installation. [\[R52\]](#)

Optional .github/workflows/training.yml - controlled manual run

```
name: Training checks
on: workflow_dispatch
jobs:
  verify:
    runs-on: [self-hosted, linux, solana-training]
    steps:
      - uses: actions/checkout@v4
      - name: Verify prepared toolchain
        run: |
          node --version
          solana --version
          anchor --version
      - name: Disposable local identities
        run: |
          mkdir -p target/deploy "$HOME/.config/solana"
          solana-keygen new --no-bip39-passphrase --silent \
            --outfile target/deploy/fixed_supply-keypair.json
          solana-keygen new --no-bip39-passphrase --silent \
            --outfile "$HOME/.config/solana/ai-coding-lab.json"
          anchor keys sync
      - name: Local validation
        run: |
          npm ci
          anchor build
          npm run typecheck
          anchor test --validator legacy
          npm --prefix app ci
          npm --prefix app run build
```

Generate and commit the root/app npm lockfiles and Cargo.lock before enabling this workflow. Use an ephemeral, isolated runner for trusted repository code and no production credentials. The CI-generated program identity is disposable for local behavior tests; a release uses its preserved deployment identity.

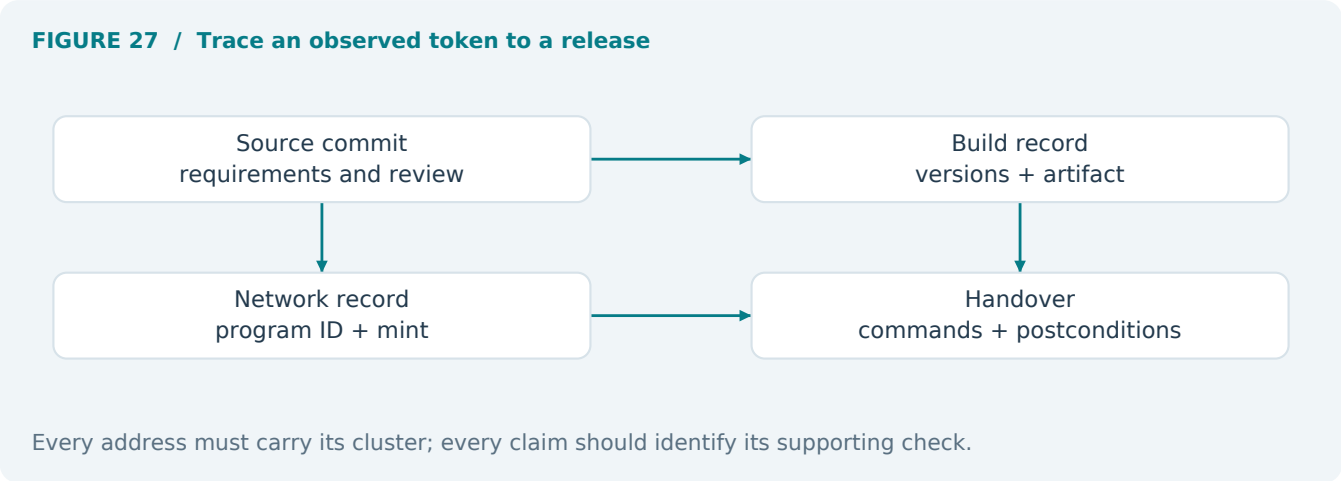
Pipeline ownership

Pin approved action revisions and toolchain versions in a maintained runner image. Report local tests, frontend build and any skipped network checks separately.

07 DELIVERY AND OPERATIONS

Release evidence and handover

Another employee should be able to identify and reproduce the intended release.



Evidence	Record
Specification	Supply policy, expected authorities, creator model and excluded features.
Repository	Source commit, reviewed diff, manifests and resolved lockfiles.
Build	Operating system, tool versions, successful commands and artifact hash.
Deployment	Cluster, program ID, deployment result and upgrade authority.
Initialization	Creator, mint, treasury, signature and decoded initial state.
Application	Frontend commit/build and configured mint address.
Operations	Known limits, troubleshooting steps and incident ownership.

For stronger provenance, use a verifiable-build workflow to compare the program artifact with the deployed executable. Anchor documents deterministic container-based builds and verification, with tooling prerequisites. Do not claim bytecode verification merely because source is public or tests passed locally. [R28]

Program upgrades and authority removal need separate records. A program made immutable cannot be updated or closed under the documented upgrade-authority rules. That decision does not replace the mint/freeze authority inspection. [R42] The training deployment may retain an upgrade authority so you can continue learning.

Handover exercise
Give a colleague the public repository and runbook. Ask them to identify the cluster, program, token mint and expected authority state before running any command. Repair the documentation where they hesitate.

07 DELIVERY AND OPERATIONS

Troubleshooting reference

Use a discriminating check before asking the assistant to change code.

Symptom	Likely layer	Check and next action
Command not found	Installation / PATH.	Reopen the correct terminal; compare the installer's PATH instructions.
Wrong Anchor import	API family.	Check 1.2.0 and @anchor-lang/core; remove obsolete @coral-xyz imports.
CPI type mismatch	Anchor migration.	Use a program public key in CpiContext::new for the selected 1.x release.
SBF program rejected locally	Build architecture.	Compare default v3 artifacts with the Agave 4.x validator requirement.
Port 8899 already in use	Validator lifecycle.	Stop the earlier local validator before Anchor-managed tests.
ConstraintSeeds	Account derivation.	Compare seed bytes, creator, program ID and client PDA.
Wrong treasury ATA	Account relationship.	Derive the ATA with the expected owner, mint and Token Program.
Insufficient SOL	Fees / storage funding.	Inspect balance and required funds on the intended test cluster.
Airdrop / HTTP 429 failure	Public service limits.	Use the official devnet faucet and bounded retry; preserve devnet.
Program missing / account not found	Cluster or address.	Check each tool's provider configuration and generated IDL address.
Mint already in use	Persistent test state.	Use an isolated fresh ledger for the initialization suite.
Token has no name / image	Metadata.	Core lab has no metadata; plan the extension before authority removal.
Frontend shows wrong mint	Build configuration.	Check app/.env.local, restart dev server or rebuild static assets.
Frontend supply unavailable	Transport / RPC / shape.	Inspect HTTP, RPC error and validated response fields separately.

For escalation, provide the command, directory, versions, cluster, public addresses, redacted error and expected outcome. Ask the assistant for a likely cause and a confirming check before editing.

Keep the invariant

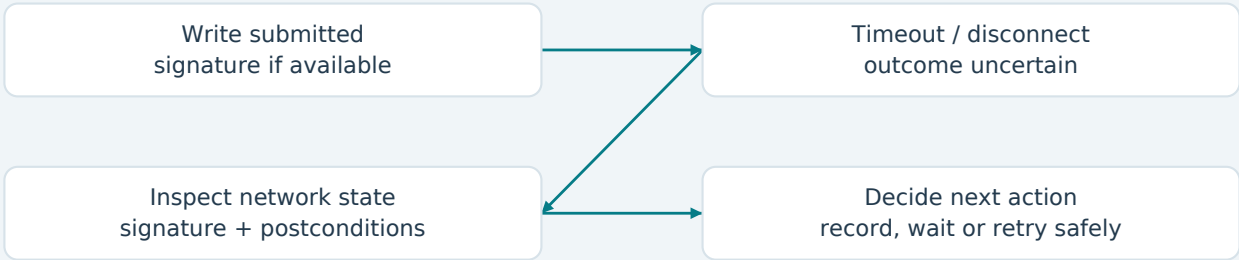
A constraint failure is useful evidence. Do not resolve it by broadening authority, changing the cluster or suppressing errors unless that change is explicitly part of a revised requirement.

07 DELIVERY AND OPERATIONS

Retries, logs and incidents

An uncertain response requires investigation before another write.

FIGURE 28 / A response is lost after a write



Repeating a transfer can move funds twice; checking existence is specific to initialization.

Retries are part of application behavior, not just error recovery. A read-only supply query can usually be retried with a limit and backoff. A signed write must consider whether the earlier attempt already landed, whether its blockhash is still valid and what repeated execution would do. The initialize program rejects the same mint's re-creation; the transfer script intentionally has no equivalent duplicate-transfer guard.

Log field	Purpose	Exclude
Timestamp and step	Order observations and find the failed operation.	Unrelated personal information.
Cluster / endpoint	Distinguish localnet and devnet results.	Credentials embedded in URLs.
Public addresses	Identify intended program, mint, treasury and recipient.	Wallet arrays or recovery phrase.
Signature / error category	Find transaction outcome and relevant logs.	Unredacted tokens or private service headers.
Versions / source commit	Reproduce the observed behavior.	Unsupported claims that unrun checks passed.

A simple incident response for the lab

Stop further writes, preserve redacted evidence and determine the actual chain state. Identify whether the issue is documentation, client behavior, program permissions or key exposure. Repair the diagnosed layer and repeat the relevant checks before resuming. If a key is exposed, replace the test identity and update the public records; do not silently continue using it.

Checkpoint - Simulate a failed RPC read and describe why a bounded retry is acceptable. Then describe how you would investigate a timed-out transfer without sending another one.

08 LEARNING AND REFERENCE

Practice exercises

Build understanding by predicting results and changing one rule at a time.

Work in a copy or branch of the local project. Predict each outcome before asking for an implementation. Run the check and explain discrepancies. Changes to issuance rules need new local mints; editing a source constant does not rewrite an existing mint.

Exercise	Task	Evidence to keep
1. Stack map	Draw which runtime executes each source file.	Correct browser / Node / Solana distinctions.
2. Prompt quality	Compare a broad request with a constrained request for one UI change.	Diff size, irrelevant changes and review effort.
3. Integer units	Convert 0.5, 25 and 1,000 tokens to raw units at six decimals.	Correct results without a floating-point intermediate.
4. Address derivation	Change the creator in a local derivation.	A different mint PDA and treasury ATA.
5. Account substitution	Run a wrong-PDA case and explain the failure.	Constraint category and uncreated intended mint.
6. Supply conservation	Transfer between two local token accounts.	Balanced deltas and unchanged supply.
7. Burn behavior	Burn a small quantity in a separate local experiment.	Reduced supply; mint authority remains absent.
8. UI failures	Test missing config, nonexistent mint and offline RPC.	Explicit errors; no false zero-supply claim.
9. Refresh behavior	Add a controlled refresh action to the app.	Disabled loading state and last-success timestamp.
10. New supply policy	Specify a capped variable-supply extension without implementing it.	Inputs, arithmetic rules and new failure cases.
11. Metadata plan	Specify metadata before authority removal.	Chosen SDK, authority and atomicity strategy.
12. Reproduction	Follow the runbook from a clean checkout.	Resolved dependencies, generated identity and passing local checks.

Compare your working methods

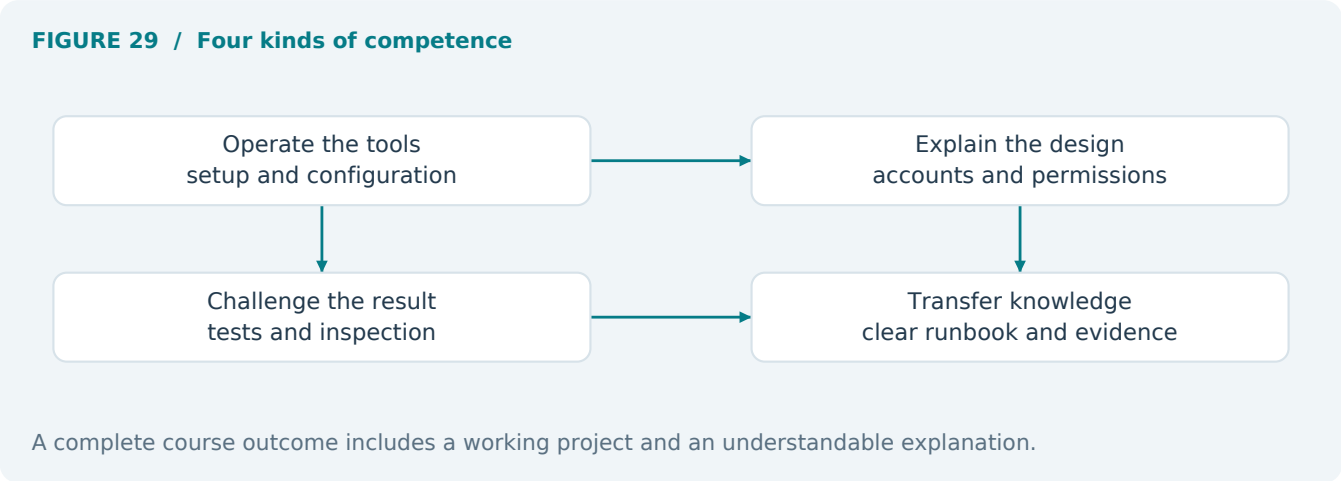
Pick two similarly sized local tasks. Record time spent understanding, prompting, editing, reviewing and validating, plus whether you can explain the finished behavior. Treat the comparison as a small personal observation rather than proof of a universal AI speed advantage. Changes in familiarity and task difficulty can dominate the apparent result.

Keep the experiment honest - Record unsuccessful attempts and review time. A generated answer that you never checked is not a completed task and should not be counted as a successful time saving.

08 LEARNING AND REFERENCE

Employee completion assessment

Demonstrate the behavior and explain why it is allowed.



Capability	Demonstration
Workstation	Show installed versions and explain the selected compatibility family.
AI task framing	Write a request with a goal, context, constraints and completion checks.
Review	Identify every permission and dependency change in a proposed diff.
Solana model	Explain program owner, token owner, mint, ATA, authority and PDA.
Implementation	Trace initialize from account validation to authority revocation.
Validation	Show six integration cases and independent decoded mint state.
Application	Show correct supply and useful error behavior from production preview.
Operations	Investigate a lost response without blindly repeating a write.

Use a simple self-assessment for each capability: 0 means unable to explain; 1 means able with help; 2 means able independently; 3 means able to demonstrate and teach it. This is a training rubric designed for the manual, not an external certification or measured proficiency scale.

Capstone handover
Produce the repository, public address record, test outputs, working dashboard and a one-page runbook. Explain two remaining limitations and one extension with its additional validation requirements.

08 LEARNING AND REFERENCE

Glossary: development

Use these terms when discussing a change with an assistant or colleague.

Term	Meaning in this manual
Acceptance criterion	A concrete observation that establishes a requirement has been met.
Agent	An assistant with tools that may inspect files, edit code and execute commands.
API	The defined interface a library or service exposes to callers.
Artifact	A produced file, such as a program binary, IDL or browser bundle.
Async / await	A JavaScript mechanism for working with operations that settle later.
Branch	A named line of development within Git.
Build	A process that transforms source into runnable or distributable output.
CI	Automated execution of agreed validation steps in a prepared environment.
Commit	A recorded snapshot of selected repository changes.
Dependency	A package or tool the project relies on.
Diff	The visible changes between two versions or states of files.
Environment variable	Process configuration; not inherently secret or authenticated.
Idempotent	Repeated application has the same intended effect as one application.
Invariant	A property that must remain true after all allowed operations.
Lockfile	The record of a resolved dependency tree.
Runtime	The environment that executes a program.
Specification	A description of required behavior, boundaries and evidence.
Stack	The cooperating tools, runtimes, libraries and services.
Type check	Static validation of relationships declared by the source's types.

Terminology is useful only when it clarifies a concrete observation. If a generated explanation says 'the stack is broken', ask which command, artifact, runtime or interface failed. If it says a change is 'secure', ask which permissions and failure cases were checked.

Useful distinction

A build proves that the source can be transformed under that toolchain. It does not prove the application's business rules, deployment configuration or network postconditions.

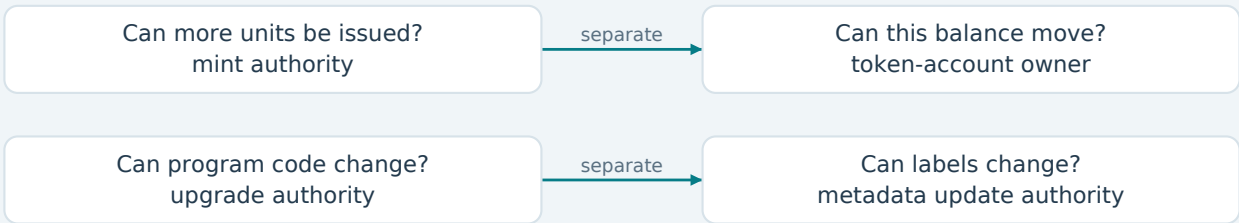
08 LEARNING AND REFERENCE

Glossary: Solana

Keep state, identity and authority separate in your vocabulary.

Term	Meaning in this project
Account	Addressed network state with lamports, data and a program owner.
ATA	The conventional derived token account for an owner, token program and mint.
Cluster	A network environment such as localnet, devnet or mainnet-beta.
CPI	A program invoking another program's instruction during execution.
Decimals	The base-10 precision used to display a raw integer token amount.
Freeze authority	Permission to freeze or thaw token accounts for a mint.
Instruction	A request to a program with accounts and encoded arguments.
Lamport	A raw SOL unit: one billion lamports equals one SOL.
Mint	The token's identity and shared state, including supply and authorities.
Mint authority	Permission to create additional units.
PDA	A deterministic program-derived address with no ordinary private key.
Program ID	The public address identifying a deployed executable.
RPC	The interface used to query network state or submit transactions.
Token account	A balance for one mint controlled by its recorded token owner.
Transaction	A signed group of instructions processed atomically.

FIGURE 30 / Four distinct authority questions



A single 'ownership revoked' label is too ambiguous for technical documentation.

08 LEARNING AND REFERENCE

References: setup and downloads

Primary project sources. All links were checked on 18 September 2026.

Use these links to install or refresh the workstation tools. The selected training versions are a documented baseline, not a promise that future latest releases share the same interfaces.

R01 | Microsoft - Visual Studio Code downloads
<https://code.visualstudio.com/download>

R02 | Git project - Install Git
<https://git-scm.com/install/>

R03 | OpenJS Foundation - Node.js downloads
<https://nodejs.org/en/download>

R04 | Node.js project - Node.js release status
<https://nodejs.org/en/about/previous-releases>

R09 | Microsoft - Install Windows Subsystem for Linux
<https://learn.microsoft.com/en-us/windows/wsl/install>

R10 | Rust project - Install Rust and rustup
<https://rust-lang.org/tools/install/>

R14 | Solana Foundation - Local installation
<https://solana.com/docs/intro/installation>

R15 | Solana Foundation - Install development dependencies
<https://solana.com/docs/intro/installation/dependencies>

R16 | Anchor project - Anchor installation and CLI introduction
<https://www.anchor-lang.com/docs/installation>

R50 | GitHub - Install the Copilot extension
<https://docs.github.com/en/copilot/how-tos/set-up/install-copilot-extension>

R59 | Solana Foundation - Solana CLI basics
<https://solana.com/docs/intro/installation/solana-cli-basics>

Download verification

Confirm the owning project, operating system and CPU architecture. Record the executable version after installation. When your organisation supplies a managed image or approved package source, use that distribution while retaining the upstream references for technical behavior.

Refresh point

Revisit the toolchain and release notes when repeating the course after a major Anchor, Node or Solana tooling update. Review the compatibility notes before replacing the selected version family.

08 LEARNING AND REFERENCE

References: languages and tooling

The language, build and source-control material behind the foundation chapters.

R05 | Microsoft - TypeScript for JavaScript programmers

<https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>

R06 | React team - React Quick Start

<https://react.dev/learn>

R07 | Vite team - Getting Started

<https://vite.dev/guide/>

R08 | Vite team - Environment variables and modes

<https://vite.dev/guide/env-and-mode>

R11 | Rust project - The Rust Book: ownership

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

R12 | Rust project - The Rust Book: data types

<https://doc.rust-lang.org/book/ch03-02-data-types.html>

R13 | Rust project - Cargo.toml versus Cargo.lock

<https://doc.rust-lang.org/cargo/guide/cargo-toml-vs-cargo-lock.html>

R47 | Git project - Pro Git: recording repository changes

<https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>

R48 | Git project - git restore reference

<https://git-scm.com/docs/git-restore>

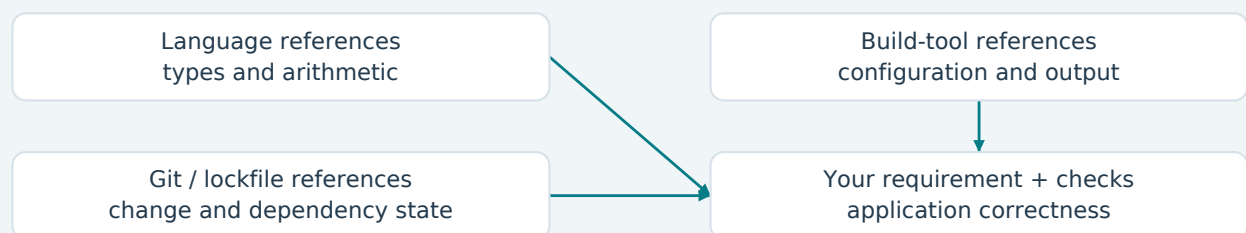
R49 | npm - npm ci reference

<https://docs.npmjs.com/cli/v11/commands/npm-ci/>

R64 | MDN Web Docs - JavaScript BigInt reference

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt

FIGURE 31 / Read the source for the failing layer



Documentation explains an interface; your acceptance criteria establish the intended behavior.

08 LEARNING AND REFERENCE

References: Solana

Token state, network interfaces, execution rules and deployment.

R29 | Solana Foundation - Solana token model

<https://solana.com/docs/tokens>

R30 | Solana Foundation - Create a token with the CLI

<https://solana.com/docs/tokens/quickstart>

R31 | Solana Foundation - Create a token mint

<https://solana.com/docs/tokens/basics/create-mint>

R32 | Solana Foundation - Create a token account

<https://solana.com/docs/tokens/basics/create-token-account>

R33 | Solana Foundation - Mint tokens

<https://solana.com/docs/tokens/basics/mint-tokens>

R34 | Solana Foundation - Transfer tokens

<https://solana.com/docs/tokens/basics/transfer-tokens>

R35 | Solana Foundation - Burn tokens

<https://solana.com/docs/tokens/basics/burn-tokens>

R36 | Solana Foundation - Set and revoke authority

<https://solana.com/docs/tokens/basics/set-authority>

R37 | Solana Foundation - Core accounts model

<https://solana.com/docs/core/accounts>

R38 | Solana Foundation - Program Derived Addresses

<https://solana.com/docs/core/pda>

R39 | Solana Foundation - Cross Program Invocation

<https://solana.com/docs/core/cpi>

R40 | Solana Foundation - Transactions and atomic execution

<https://solana.com/docs/core/transactions>

R41 | Solana Foundation - Fees and compute limits

<https://solana.com/docs/core/fees>

R42 | Solana Foundation - Deploying and upgrading programs

<https://solana.com/docs/programs/deploying>

R43 | Solana Foundation - JSON-RPC overview

<https://solana.com/docs/rpc>

R61 | Solana Foundation - Transaction structure and format limits

<https://solana.com/docs/core/transactions/transaction-structure>

R62 | Solana Foundation - Fee structure and priority-fee formulas

<https://solana.com/docs/core/fees/fee-structure>

R65 | Solana Foundation - getTokenSupply RPC method

<https://solana.com/docs/rpc/http/gettokensupply>

Format note: the course's operator client uses legacy / v0-compatible web3.js v1 interfaces. Current Solana references also discuss Kit and newer v1 transaction formats. Those examples must not be copied into the lab without a deliberate client migration.

08 LEARNING AND REFERENCE

References: Anchor

Release-sensitive program, client and validation references.

R17 | Anchor project - Anchor 1.2.0 release notes

<https://www.anchor-lang.com/docs/updates/release-notes/1-2-0>

R18 | Anchor project - Anchor 1.0.0 migration and breaking changes

<https://www.anchor-lang.com/docs/updates/release-notes/1-0-0>

R19 | Anchor project - TypeScript client

<https://www.anchor-lang.com/docs/clients/typescript>

R20 | Anchor project - Anchor CLI reference

<https://www.anchor-lang.com/docs/references/cli>

R21 | Anchor project - Anchor.toml configuration

<https://www.anchor-lang.com/docs/references/anchor-toml>

R22 | Anchor project - Account constraints

<https://www.anchor-lang.com/docs/references/account-constraints>

R23 | Anchor project - Program IDL file

<https://www.anchor-lang.com/docs/basics/idl>

R24 | Anchor project - Minting tokens through CPI

<https://www.anchor-lang.com/docs/tokens/basics/mint-tokens>

R25 | Anchor project - Creating token accounts

<https://www.anchor-lang.com/docs/tokens/basics/create-token-account>

R26 | Anchor project / Rustdoc - anchor-spl token module API

https://docs.rs/anchor-spl/latest/anchor_spl/token/index.html

R27 | Anchor project / Rustdoc - set_authority API

https://docs.rs/anchor-spl/latest/anchor_spl/token/fn.set_authority.html

R28 | Anchor project - Verifiable builds

<https://www.anchor-lang.com/docs/references/verifiable-builds>

R57 | Anchor project - Local development walkthrough

<https://www.anchor-lang.com/docs/quickstart/local>

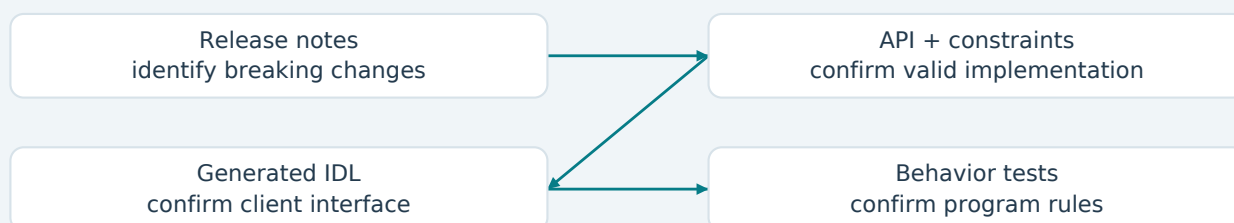
R58 | Anchor project - Quickstart entry point

<https://www.anchor-lang.com/docs/quickstart>

R60 | Solana Foundation - Anchor CLI basics

<https://solana.com/docs/intro/installation/anchor-cli-basics>

FIGURE 32 / The reference sequence for an Anchor change



Rustdoc /latest links can change: select 1.2.0 when checking this edition's code.

08 LEARNING AND REFERENCE

References: workflow and evidence

Metadata, client helpers, AI tools, security and productivity research.

R44 | Metaplex - Token Metadata overview

<https://www.metaplex.com/docs/smart-contracts/token-metadata>

R45 | Solana SPL project - getOrCreateAssociatedTokenAccount API

<https://solana-labs.github.io/solana-program-library/token/js/functions/getOrCreateAssociatedTokenAccount.html>

R46 | Solana SPL project - transferChecked API

<https://solana-labs.github.io/solana-program-library/token/js/functions/transferChecked.html>

R51 | GitHub - About Copilot cloud agent

<https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent>

R52 | GitHub - Building and testing Node.js with Actions

<https://docs.github.com/en/actions/tutorials/build-and-test-code/nodejs>

R53 | OWASP - LLM01:2025 Prompt Injection

<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

R54 | Peng et al. - The Impact of AI on Developer Productivity (2023)

<https://arxiv.org/abs/2302.06590>

R55 | METR - Early-2025 experienced developer productivity study (10 July 2025)

<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>

R56 | METR - Developer productivity experiment update (24 February 2026)

<https://metr.org/blog/2026-02-24-uplift-update/>

R63 | Metaplex - Minting assets and metadata creation

<https://www.metaplex.com/docs/smart-contracts/token-metadata/mint>

What is original to this manual

The training token specification, source scaffold, diagrams, practice exercises, learning allowance and assessment rubric were authored for this document. Numerical token examples are calculated from the chosen six-decimal supply. They are not observations from a deployed commercial token.

How the sources were used

Official project documentation establishes installation paths and API behavior. Language references support types and arithmetic. The research papers and METR updates support the limited statistical examples on page 6. The manual does not infer a universal productivity rate or token-market forecast.

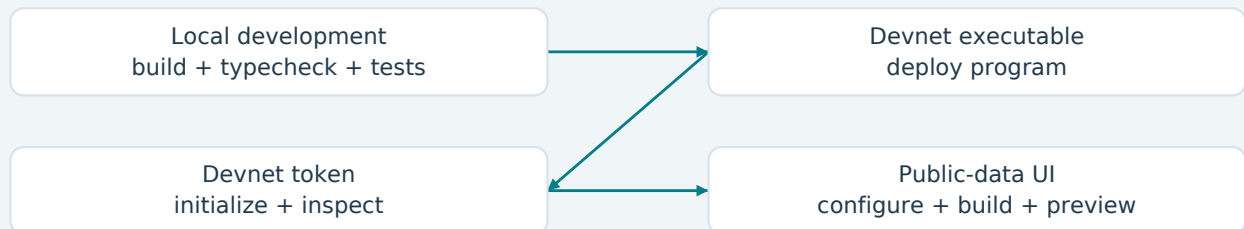
External websites, installers, package versions, public faucets and RPC behavior can change. Keep your exact resolved versions and executed validation results alongside the document.

08 LEARNING AND REFERENCE

Quick reference

The compact runbook after you understand the full workflow.

FIGURE 33 / Select the right operation



Stop at each failed gate; investigate the layer before continuing.

Project root - local acceptance

```
anchor build
npm run typecheck
anchor test --validator legacy
```

Project root - devnet; wallet and program IDs must already be checked

```
anchor deploy --provider.cluster devnet
export ANCHOR_PROVIDER_URL=https://api.devnet.solana.com
export ANCHOR_WALLET="$HOME/.config/solana/ai-coding-lab.json"
npm run initialize
npm run inspect
```

Project root - dashboard; configure app/.env.local first

```
npm --prefix app run dev
npm --prefix app run build
npm --prefix app run preview
```

Five facts to record

Cluster, source commit, program ID, mint address and creator/treasury addresses. Keep successful check outputs and transaction signatures beside them. Keep private keys outside the repository.

For the next feature, write the acceptance criteria first, give the assistant a bounded request, review its diff and execute the relevant checks. Preserve a checkpoint once the evidence matches the intended behavior.